



**Hochschule
Bonn-Rhein-Sieg**

*University
of Applied Sciences*

Fachbereich Informatik
Department of Computer Sciences

Abschlussarbeit

Studiengang Bachelor Informatik

Entwurf und Evaluation eines fehlertoleranten Job-Processing in Java

von

Max Urfei

Erstprüfer: Prof. Dr. Andreas Hackelöer

Zweitprüfer: M. Sc. Moritz Balg

eingereicht am TT.MM.YYYY

Inhaltsverzeichnis

Abkürzungsverzeichnis	iii
Abbildungsverzeichnis	iv
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung	2
1.4 Hypothesen	3
1.5 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Grundlagen der Hintergrundverarbeitung	4
2.1.1 Asynchrone Verarbeitung und Queueing	4
2.1.2 Ausführungssemantiken	6
2.1.3 Background-Job-Processing	7
2.2 Konsistenz und konkurrierende Verarbeitung	8
2.2.1 ACID	9
2.2.2 Transaktionsgrenzen	10
2.2.3 Synchronisationsverfahren und Isolationsstufen	11
2.3 Fehlertoleranz und Wiederherstellung	12
2.3.1 Fehlerklassifikation	12
2.3.2 Retry-/Backoff-Strategien	13
2.3.3 Recovery	15
2.3.4 Idempotenz	16
2.4 Stand der Forschung	17
2.4.1 Brokerbasierte Ansätze	17
2.4.2 Datenbankbasierte Ansätze	18
2.4.3 Einordnung und Forschungslücke	18
3 Methodik	20
3.1 Literaturrecherche	20
3.2 Ergebniserzeugung	20
4 Anforderungen	21
4.1 Systemkontext	21
4.2 Abgrenzung des betrachteten Fehlermodells	22
4.3 Zusicherungen	24
4.3.1 Joberstellung	25
4.3.2 Jobverarbeitung	25
4.3.3 Recovery	26

5	Umsetzung	28
5.1	Architektur und Design	28
5.1.1	Architekturentwurf	28
5.1.2	Zustandsmodell	30
5.1.3	Entwurf der Datenbank	31
5.1.4	Joberstellung	32
5.1.5	Exklusive Verarbeitung und Transaktionsmodell	34
5.1.6	Retry-Strategie	36
5.1.7	Recovery	37
5.1.8	Idempotente Ergebniserzeugung	39
5.2	Implementierung	40
5.2.1	Technologie-Stack	40
5.2.2	Persistenz und Datenbankzugriff	40
5.2.3	Joberstellung	42
5.2.4	Claiming und Verarbeitung	44
5.2.5	Abschluss einer Jobverarbeitung	46
5.2.6	Retries	48
5.2.7	Recovery	50
6	Evaluation	52
6.1	Fehlerszenarien	52
6.2	Durchführung	52
6.3	Ergebnisse	52
7	Diskussion	53
7.1	Gewonnene Erkenntnisse	53
7.2	Einschränkungen und Grenzen der Arbeit	53
8	Fazit und Ausblick	55
	Literaturverzeichnis	56

Abkürzungsverzeichnis

ACID Atomicity, Consistency, Isolation, Durability. 9–11, 18, 28, 31, 34, 36

API Application Programming Interface. 29, 33, 36

CRUD Create, Read, Update, Delete. 1

DTO Data Transfer Object. 42

REST Representational State Transfer. 7, 21, 25, 29, 30, 32, 33, 36, 42, 48, 50

UUID Universally Unique Identifier. 16, 40, 42, 44

Abbildungsverzeichnis

2.1	Vergleich synchroner und asynchroner Kommunikation	5
2.2	Queueing	5
2.3	Verhalten bei transienten Fehlern	14
4.1	Systemkontext	21
4.2	Übersicht betrachteter Fehlerklassen	23
4.3	Systemanforderungen	25
5.1	Abstrakte Architektur des entwickelten Systems	29
5.2	Zustandsmodell eines Jobs	30
5.3	Datenbankschema des entwickelten Job-Processing-Systems	31
5.4	Ablauf der Jobannahme	33
5.5	Transaktionsmodell	35
5.6	Klassendiagramm der Retry-Komponente	37
5.7	Mehrfachausführung mit Seiteneffekten	39

1 Einleitung

1.1 Motivation

In modernen Softwaresystemen stellt die Verarbeitung von Hintergrundjobs einen zentralen Bestandteil vieler Backend-Architekturen dar (Kleppmann 2017, Kap. 10). Zahlreiche Geschäftsprozesse, darunter die Generierung von Reports, der Import großer Datenmengen, periodische Abrechnungsläufe oder das Versenden asynchroner Benachrichtigungen, erfordern eine Verarbeitung, die entkoppelt vom synchronen Request-Response-Zyklus stattfindet. Insbesondere in industrienahen und sicherheitskritischen Anwendungen, etwa im Bereich der luftfahrttechnischen Wartungsdokumentation, müssen solche Prozesse nicht nur funktional korrekt ablaufen, sondern darüber hinaus zuverlässig, nachvollziehbar und fehlertolerant sein. Aber auch in nicht-kritischen Systemen ist Zuverlässigkeit von Bedeutung für die Nutzerfreundlichkeit der Anwendung, als auch um Produktivitäts- und Gewinneinbußen in Produktivsystemen in Folge von Fehlern zu vermeiden (Kleppmann 2017, Kap. 1 Abschnitt „How Important Is Reliability?“).

1.2 Problemstellung

In der Praxis können bei der Verarbeitung von Hintergrundjobs jederzeit Teilausfälle auftreten. Prozesse können während der Verarbeitung unerwartet terminieren, Datenbankverbindungen können kurzzeitig unterbrochen werden, identische Anfragen infolge von clientseitigen Wiederholungsversuchen mehrfach an das System übermittelt werden oder Ergebnisse bzw. Antworten gehen verloren (Kleppmann 2017, Kap. 8). Ohne geeignete Mechanismen führen derartige Szenarien zu inkonsistenten Systemzuständen. Beispielsweise verbleiben Jobs im Status „laufend“, obwohl kein Worker sie mehr aktiv verarbeitet, Ergebnisse werden doppelt erzeugt oder Aufträge gehen vollständig verloren.

Bestehende Frameworks und Bibliotheken, etwa Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026b), stellen umfangreiche Funktionalitäten für die Hintergrundverarbeitung bereit. Hierzu zählen unter anderem Scheduling, Wiederholungsmechanismen sowie Verfahren zur Fehlerbehandlung und Wiederaufnahme. Sie verfolgen dabei jedoch jeweils eigene Architekturansätze und abstrahieren die zugrunde liegenden Mechanismen weitgehend hinter ihren Programmierschnittstellen. Dadurch eignen sie sich nur eingeschränkt, um systematisch zu untersuchen, welche Architekturentscheidungen für bestimmte Konsistenz- und Verarbeitungszusicherungen tatsächlich erforderlich sind.

Die beschriebene Problematik ist von hoher praktischer und industrieller Relevanz. Nahezu jedes Backend-System, das über einfache Create, Read, Update, Delete (CRUD)-Operationen hinausgeht, muss Aufgaben zuverlässig im Hintergrund verarbeiten und dabei konsistente Datenbestände gewährleisten (Richardson 2018, Kap. 4). Fehler in diesem Bereich führen zu Datenverlust, inkonsistenten Geschäftszuständen sowie unnötigem manuellem Eingriff und können insbesondere in regulierten Domänen wie der Luft-

fahrt Compliance-Verstöße nach sich ziehen. Auch im Kontext von Cloud-nativen Architekturen ist dieses Thema relevant. Horizontale Skalierung, Container-Neustarts und kurzzeitige Netzwerkunterbrechungen gehören dort zum regulären Betriebsverhalten und müssen bereits beim Entwurf eines Systems berücksichtigt werden. Daher bilden die in dieser Arbeit behandelten Konzepte auch die Grundlage für den zuverlässigen Betrieb von Backend-Systemen in Cloud-Umgebungen.

Obwohl die zugrundeliegenden Konzepte wie Fehlertoleranz, Recovery, Idempotenz, Nebenläufigkeitskontrolle und Transaktionsmanagement in der Literatur umfassend beschrieben werden, fehlen Arbeiten, die diese Aspekte zu einer durchgängigen datenbank-basierten Architektur für Background-Job-Processing zusammenführen und deren Verhalten unter definierten Fehlerszenarien systematisch evaluieren. Ebenso implementieren bestehende Frameworks diese Mechanismen häufig als interne Abstraktionen, wodurch deren Zusammenspiel und Auswirkungen auf die Zuverlässigkeit des Gesamtsystems für Entwickler nur eingeschränkt nachvollziehbar sind.

Daraus ergibt sich die dieser Arbeit zugrundeliegende Hauptforschungsfrage:
Welche Architekturmechanismen und Entwurfsentscheidungen sind notwendig, um in einem Java/Spring-basierten Job-Processing-Backend definierte Konsistenzzusicherungen unter typischen Teilausfallszenarien nachweisbar einzuhalten?

Zur Beantwortung dieser Frage werden folgende Teilfragen untersucht:

- TF1: Welche Zusicherungen muss das System gewährleisten, damit Jobs stets korrekt und ohne Inkonsistenzen verarbeitet werden?
- TF2: Welchen Beitrag leisten Idempotenz, Retry-Strategien und Recovery-Mechanismen zur Robustheit unter definierten Fehlerszenarien?
- TF3: Wie müssen Transaktionen und Locking-Strategien genutzt werden, damit bei paralleler Arbeit mehrerer Worker keine Doppelverarbeitung oder inkonsistenten Zustände entstehen?
- TF4: Wie lässt sich sicherstellen, dass Jobs nach einem unerwarteten Prozessabbruch korrekt wiederaufgenommen werden, ohne dass Aufträge verloren gehen?

1.3 Zielsetzung

Ziel dieser Arbeit ist der Entwurf, die prototypische Implementierung und die Evaluation eines fehlertoleranten Job-Processing-Backends auf Basis einer relationalen Datenbank in Kombination mit Java und Spring. Dabei soll herausgearbeitet werden, welche Architekturmechanismen erforderlich sind, damit Hintergrundjobs auch bei Teilausfällen zuverlässig und korrekt verarbeitet werden und definierte Konsistenzzusicherungen eingehalten werden.

Konkret verfolgt die Arbeit folgende Teilziele:

- TZ1: Definition von Zusicherungen: Es sollen präzise Korrektheitsbedingungen formuliert werden, die das System zu jedem Zeitpunkt einhalten muss, unabhängig davon,

welche Fehler auftreten. Dazu zählt insbesondere, dass kein Job doppelt erfolgreich abgeschlossen wird und dass kein Job dauerhaft verloren geht.

- TZ2: Architekturentwurf: Es soll eine Architektur entworfen werden, die die hergeleiteten Zusicherungen umsetzt. Architekturentscheidungen sollen explizit dokumentiert und begründet werden.
- TZ3: Prototypische Implementierung: Die entworfene Architektur soll als funktionsfähiger Prototyp umgesetzt werden, der die definierten Zusicherungen unter realistischen Bedingungen einhält.
- TZ4: Evaluation unter Fehlerbedingungen: Es soll nachgewiesen werden, dass das System seine Zusicherungen auch bei gezielt herbeigeführten Teilausfällen nicht verletzt.

1.4 Hypothesen

Auf Basis der Forschungsfrage und der Zielsetzung werden folgende Hypothesen formuliert, deren Überprüfung im Rahmen der Evaluation erfolgt.

- H1: Durch die Kombination eines persistenten Statusmodells mit transaktionsbasiertem Locking und Idempotenz-Mechanismen lässt sich ein System entwerfen, das auch bei unerwarteten Prozessabbrüchen keine Jobs verliert und keine doppelten Ergebnisse erzeugt.
- H2: Retry-Strategien mit exponentiellem Backoff in Verbindung mit einer Fehlerklassifikation (transient vs. permanent) erhöhen die Robustheit des Systems gegenüber kurzzeitigen Infrastrukturausfällen, ohne dabei die definierten Zusicherungen zu verletzen.
- H3: Die definierten Zusicherungen lassen sich durch automatisierte Tests mit gezielter Fehlerinjektion reproduzierbar überprüfen und nachweisen.

1.5 Aufbau der Arbeit

To be done

2 Grundlagen

Dieses Kapitel vermittelt die theoretischen Grundlagen, die für das Verständnis der im weiteren Verlauf entwickelten Architektur erforderlich sind. Zunächst werden die grundlegenden Konzepte der Hintergrundverarbeitung sowie die Anforderungen an zuverlässige Background-Job-Processing-Systeme erläutert. Darauf aufbauend werden die für diese Arbeit relevanten Mechanismen zur Gewährleistung von Konsistenz bei paralleler Verarbeitung sowie Konzepte zur Fehlertoleranz, Wiederherstellung und idempotenten Verarbeitung eingeführt. Die dargestellten Grundlagen bilden die Basis für den in Kapitel 5.1 vorgestellten Architekturentwurf und dessen anschließende Implementierung und Evaluation.

2.1 Grundlagen der Hintergrundverarbeitung

Hintergrundverarbeitung dient dazu, zeitintensive oder ressourcenintensive Aufgaben vom synchronen Request-Response-Zyklus zu entkoppeln. Gleichzeitig entstehen daraus jedoch auch einige Herausforderungen: In verteilten Umgebungen sind Teilausfälle, Timeouts und kurzzeitige Unterbrechungen üblich, sodass ohne entsprechende Mechanismen nicht davon ausgegangen werden kann, dass eine angestoßene Verarbeitung vollständig und genau einmal ausgeführt wird. Insbesondere kann bei Kommunikationsabbrüchen oder Wiederholungsversuchen unklar bleiben, ob eine Operation bereits verarbeitet wurde oder ob sie erneut angestoßen werden muss (Kleppmann 2017, Kap. 8). Vor diesem Hintergrund werden in den folgenden Abschnitten grundlegende Konzepte vorgestellt, die eine Entkopplung der Verarbeitung ermöglichen und die Basis für robuste Hintergrundverarbeitung bilden.

2.1.1 Asynchrone Verarbeitung und Queueing

In Backend-Systemen lässt sich Kommunikation und Verarbeitung bezüglich der zeitlichen Kopplung zwischen synchronem und asynchronem Verhalten unterscheiden. Der Unterschied der Funktionsweise wird in Abbildung 2.1 deutlich. Bei synchroner Verarbeitung nach dem Request-Response-Stil wartet der Aufrufer auf eine Antwort auf die Anfrage und blockiert gegebenenfalls, bevor er fortfährt. Bei asynchroner Verarbeitung wird die Antwort vom Zeitpunkt der Auftragserstellung entkoppelt. Der Aufrufer initiiert die Arbeit, blockiert jedoch nicht und kann auch während des Wartens auf eine Antwort weiter arbeiten (Richardson 2018, Kap. 3.1.1).

Zur praktischen Umsetzung asynchroner Verarbeitung werden häufig Puffer eingesetzt. Dieses Prinzip wird als Queueing bezeichnet. Dadurch können Nachrichtenverluste aufgrund von Überlastung reduziert und Lastspitzen ausgeglichen werden, indem der Puffer erhaltene Anfragen zur späteren Ausführung zwischenspeichert (Kleppmann 2017, Kap. 11, Abschnitt „Message brokers“). Das grundlegende Prinzip ist in Abbildung 2.2 schemenhaft dargestellt. Ein solcher Puffer kann verschiedene Ausprägungen annehmen, beispielsweise als In-Memory beim Empfänger, als Datenbank oder in Form eines Messa-

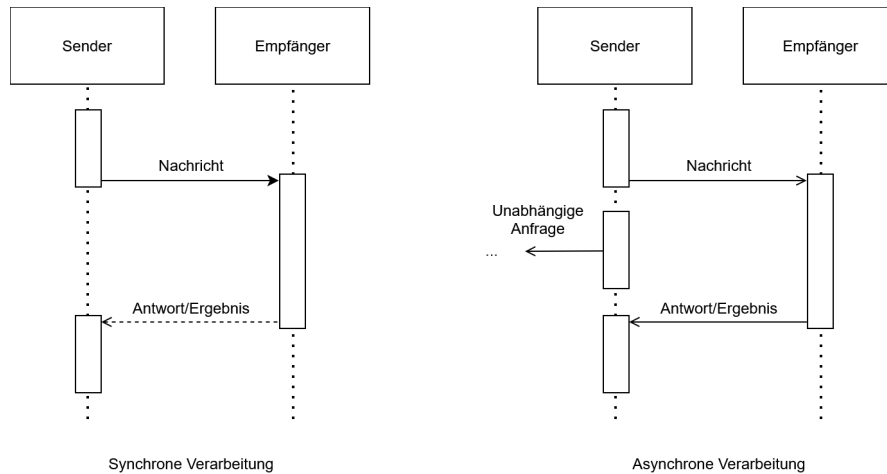


Abbildung 2.1: Vergleich synchroner und asynchroner Kommunikation

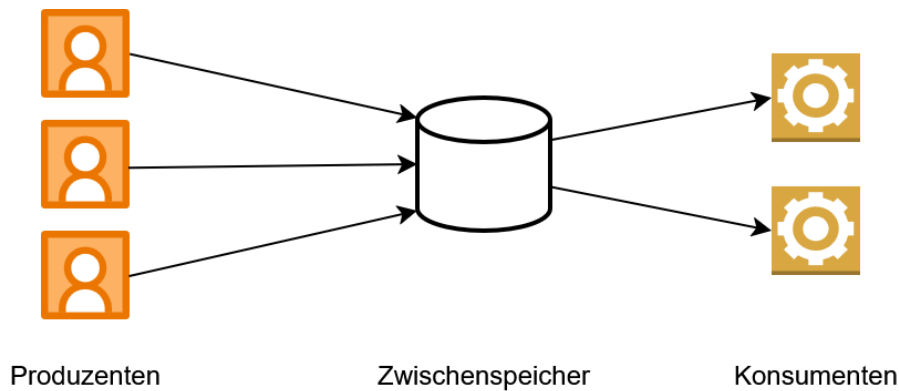


Abbildung 2.2: Queueing

ge Brokers. Ein Produzent sendet eine Nachricht an einen Zwischenspeicher und wartet in der Regel lediglich auf das Acknowledgment, dass die Nachricht zwischengespeichert wurde, nicht jedoch auf die Verarbeitung oder das Ergebnis (Kleppmann 2017, Kap. 11, Abschnitt „Messaging Systems“). Die so erreichte Entkopplung hat gleichzeitig den Vorteil, dass die Verarbeitung der Anfrage unabhängig von der Verbindung zwischen Produzent und Konsument wird. Im Gegensatz dazu kann bei synchronen Systemen bei Verbindungsabbrüchen oder Timeouts unklar bleiben, ob eine Anfrage den Server erreicht hat oder bereits verarbeitet wurde. Genau dieses Problem adressieren Message Broker: Falls ein Konsument temporär nicht verfügbar ist oder während der Verarbeitung ausfällt, können Message Broker trotzdem für die At-Least-Once-Semantik (vgl. Kapitel 2.1.2) sorgen (Kleppmann 2017, Kap. 11, Abschnitt „Acknowledgments and redelivery“). Des Weiteren müssen Verarbeitungskapazitäten in Folge von Queueing nicht auf die maximal zu erwartende Last ausgelegt werden, sondern können sich an der durchschnittlichen

Auslastung orientieren. Kurzzeitige Lastspitzen werden dabei durch die Zwischenspeicherung der Aufgaben in der Queue ausgeglichen (Microsoft Corporation (Hrsg.) 2026b). Die synchrone Request-Response-Kommunikation bietet demgegenüber den Vorteil einer unmittelbaren Rückmeldung über Erfolg oder Fehler einer Operation, setzt jedoch voraus, dass Client und Server während der gesamten Interaktion verfügbar sind (Richardson 2018, Kap. 3.4).

Die asynchrone Verarbeitung bildet damit die Grundlage für die Hintergrundverarbeitung, indem die Entkopplung genutzt wird, um Aufgaben unabhängig vom ursprünglichen Aufruf auszuführen und gleichzeitig die Robustheit und Skalierbarkeit des Systems zu erhöhen (Newman 2026, Kap. 8, Abschnitt „Why Use A Message Broker?“).

2.1.2 Ausführungssemantiken

In verteilten Systemen führen potenzielle Prozessabstürze und Teilausfälle dazu, dass Aufträge verloren gehen oder der Verarbeitungszustand einer Aufgabe unklar sein kann. Um zu beschreiben, was Systeme aufgrund ihres Entwurfs und ihrer Mechanismen zur Fehlertoleranz bezüglich Auslieferung beziehungsweise Ausführung garantieren, werden grundsätzlich drei fundamentale Ausführungssemantiken unterschieden, die Newman (2026, Kap. 8 Abschnitt „Types of Delivery Guarantees“) vor dem Hintergrund der Auslieferung wie folgt definiert. Im Kontext von Job-Processing sind diese Semantiken zur Nachrichtenzustellung analog auf die Ausführung der Jobs anwendbar.

At-Least-Once

Ein Konsument erhält die Nachricht mindestens einmal. Bei der Auslieferung bestätigt der Konsument über ein Acknowledge das Ankommen der Nachricht. Sofern das Acknowledgement beim Absender eingeht, ist für diesen garantiert, dass die Nachricht angekommen ist. Bleibt diese Bestätigung innerhalb eines definierten Zeitfensters jedoch aus, z.B. weil die Nachricht oder das Acknowledgement verloren geht, wird die Nachricht erneut zugestellt. Dieser Fall wird Redelivery genannt. Dieses Verfahren verhindert Datenverlust, birgt jedoch das Risiko von Duplikaten, für den Fall, dass die Nachricht ankommt, aber das Acknowledge verloren geht und es infolgedessen zur Redelivery kommt. Einschränkung sei hier gesagt, dass die Zustellung nicht garantiert werden kann, wenn der Konsument bei keinem Zustellversuch verfügbar ist.

At-Most-Once

Ein Konsument erhält die Nachricht maximal einmal. Der Produzent schickt die Nachricht ab und erwartet keine Empfangsbestätigung. Eine erneute Zustellung findet unter keinen Umständen statt. Dadurch werden Duplikate strikt ausgeschlossen, jedoch muss ein potenzieller Datenverlust zugunsten besserer Latenzen hingenommen werden.

Exactly-Once

Diese Semantik garantiert, dass eine Nachricht genau einmal beim Konsumenten ankommt. In der Praxis ist das nur schwer zu realisieren, da beispielsweise nicht klar sein kann, ob ein Acknowledgement nur noch nicht angekommen, oder schon verloren gegangen ist. Viele Umsetzungen beruhen daher auf der Notwendigkeit, dass sowohl beim Produzenten, als auch Konsumenten entsprechende Mechanismen implementiert werden. Eine Variante zur Umsetzung von Exactly-Once ist die Approximation über eine Kombination der Implementierung von At-Least-Once beim Produzent und Idempotenz beim Konsument. Die Garantie bezieht sich somit nicht auf die tatsächliche Anzahl der Zustellungen einer Nachricht, sondern auf die Berücksichtigung durch den Konsumenten. Eine Nachricht kann mehrfach zugestellt werden, solange sie nur einmal angenommen wird.

2.1.3 Background-Job-Processing

Aufbauend auf den Konzepten der asynchronen Verarbeitung (vgl. Kapitel 2.1.1) bezeichnet das Background-Job-Processing die Ausführung von Aufgaben außerhalb des Request-Response-Zyklus. Hierbei werden fachliche Operationen als eigenständige Arbeitseinheiten (Jobs) modelliert und zeitlich entkoppelt verarbeitet.

In der Softwarearchitektur existieren verschiedene etablierte Entwurfsmuster zur Umsetzung einer solchen Infrastruktur, wie beispielsweise das Queue-Based Load Leveling Pattern (Microsoft Corporation (Hrsg.) 2026b). Dieses Muster besteht ähnlich des Konzepts aus Abbildung 2.2 aus drei funktionalen Kernkomponenten:

- **Der Produzent:** Erzeugt einen Job und gibt diesen weiter an den Job-Store. Das kann zum einen direkt durch einen Service geschehen, oder beispielsweise auch über den Aufruf eines entfernten Endpoints, der den Job über eine synchrone Schnittstelle, wie z.B. Representational State Transfer (REST), entgegennimmt. In letzterem Fall fungiert der Service hinter dem Endpoint als Produzent. Dessen Aufgabe besteht darin, den Auftrag zu validieren und an die Persistenzschicht zu übergeben, woraufhin der synchrone Request-Response-Zyklus für den Client sofort beendet wird. Jobs können aber nicht nur durch solche Event-driven Trigger ausgelöst werden, sondern auch durch Schedule-driven Trigger. Dabei erfolgt die Ausführung eines Jobs periodisch oder zu vordefinierten Zeitpunkten, z.B. über durch einen Cron-Trigger (Microsoft Corporation (Hrsg.) 2026a).
- **Der Job-Store:** Speichert die Jobs für den Zeitraum von der Erstellung bis mindestens zur endgültigen Beendigung zwischen. Für die Umsetzung existieren verschiedene Varianten, beispielsweise als Message Broker oder relationale Datenbank. Diese Realisierungsmöglichkeiten bieten jeweils spezifische Vor- und Nachteile in der Nutzung. Die Auswahl des passenden Ansatzes sollte daher durch die Qualitätsanforderungen an das System, wie z.B. Durchsatz oder Akzeptanz einzelner verlorener Jobs, begründet werden (Kleppmann 2017, Kap. 11).
- **Der Konsument:** Autonome Komponente, die die Fachlogik des hinterlegten Jobs ausführt. Sobald der Konsument über freie Systemressourcen verfügt, kann er einen

wartenden Job aus dem Job-Store beanspruchen und führt diesen aus (Microsoft Corporation (Hrsg.) 2026b).

Durch eine solche Architektur können Background-Jobs als „fire and forget“-Operationen (Microsoft Corporation (Hrsg.) 2026a) implementiert werden. Sie laufen asynchron in einem vom Client isolierten Prozess, sodass deren Verarbeitungszeit keine unmittelbare Auswirkung auf die Reaktionszeit oder Verfügbarkeit des Produzenten hat (Microsoft Corporation (Hrsg.) 2026a). Außerdem ermöglicht dieses Konzept das Abfangen von Lastspitzen und flexiblere Skalierung (Microsoft Corporation (Hrsg.) 2026b).

Die Zuweisung von Jobs an Konsumenten kann grundsätzlich über unterschiedliche Modelle erfolgen. Beim Push-Modell weist eine zentrale Komponente Aufgaben aktiv freien Konsumenten zu. Beim Pull-Modell fragen Konsumenten eigenständig neue Aufgaben aus dem Job-Store ab. Beide Ansätze unterscheiden sich hinsichtlich Komplexität, Skalierbarkeit und Koordinationsaufwand (Kleppmann 2017, Kap. 11 Abschnitt „Transmitting Event Streams“).

Werden mehrere Konsumenten parallel betrieben, entsteht die Herausforderung der konkurrierenden Verarbeitung. Ohne geeignete Koordinationsmechanismen könnten mehrere Instanzen denselben Job gleichzeitig beanspruchen oder widersprüchliche Statusänderungen durchführen. Die hierfür relevanten Konzepte werden in Kapitel 2.2 behandelt.

Da die Verarbeitung zeitlich entkoppelt erfolgt, müssen Informationen über offene und bereits bearbeitete Jobs über den Lebenszyklus einzelner Prozesse hinweg erhalten bleiben. Hierfür ist eine persistente Speicherung des Verarbeitungszustands erforderlich. Insbesondere in verteilten Systemen können Teilausfälle auftreten, bei denen einzelne Komponenten oder Prozesse während der Ausführung ausfallen, während andere Teile des Systems weiterhin verfügbar bleiben. Damit eine unterbrochene Verarbeitung nach einem solchen Ausfall wieder aufgenommen werden kann, muss das System den zuletzt bekannten Zustand kennen oder anhand persistenter Informationen rekonstruieren können (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“).

Neben Prozessabstürzen können auch externe Dienste vorübergehend nicht erreichbar sein oder Kommunikationsfehler auftreten. Das System muss daher abhängig von der jeweiligen Fehlersituation entscheiden können, ob eine Verarbeitung erneut durchgeführt, fortgesetzt oder endgültig beendet werden soll (Microsoft Corporation (Hrsg.) 2026a). Die hierfür relevanten Konzepte zur Fehlerbehandlung und Wiederherstellung werden in Kapitel 2.3 vorgestellt.

Da sowohl Wiederholungsversuche als auch Wiederaufnahmen nach einem Ausfall dazu führen können, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden, muss darüber hinaus sichergestellt werden, dass solche Mehrfachausführungen keine inkonsistenten Ergebnisse verursachen. Hierzu werden Verfahren der idempotenten Verarbeitung eingesetzt, die in Kapitel 2.3.4 vorgestellt werden.

2.2 Konsistenz und konkurrierende Verarbeitung

Bei der Verarbeitung von Daten durch mehrere parallel arbeitende Prozesse entsteht die Herausforderung, dass konkurrierende Zugriffe auf gemeinsame Ressourcen zu inkon-

sistenten Zuständen führen können. Ohne geeignete Mechanismen können Änderungen verloren gehen, widersprüchliche Zustände entstehen oder mehrere Prozesse dieselben Daten gleichzeitig verändern (Saake, Heuer und Sattler 2018, Kap. 13.1.2).

Im Kontext von Background-Job-Processing tritt dieses Problem besonders dann auf, wenn mehrere Worker-Instanzen parallel auf denselben Jobbestand zugreifen. Versuchen sie ohne geeignete Synchronisationsmechanismen denselben Job zu beanspruchen oder konkurrierende Statusänderungen durchzuführen, können Race Conditions entstehen. Insbesondere bei Pull-basierten Architekturen, bei denen die Jobverteilung nicht durch eine Orchestrierungskomponente realisiert wird, muss eine solche ungewollte Doppelbeanspruchung explizit verhindert werden (Kleppmann 2017, Kap. 7 Abschnitt „Isolation“). Die Gewährleistung konsistenter Zustände stellt daher eine zentrale Anforderung an Systeme mit paralleler Verarbeitung dar. Die hierfür etablierten Konzepte und Mechanismen werden in den folgenden Abschnitten vorgestellt.

2.2.1 ACID

Zur Vereinfachung der Gewährleistung konsistenter Zustände bei parallelen Zugriffen existiert das Konzept der Transaktionen. Diese fassen mehrere Operationen zu einer logischen Einheit zusammen, deren Ausführung nach außen als zusammenhängender Vorgang erscheint (Kleppmann 2017, Kap. 7). Im Kontext persistenter Datenhaltung werden von Transaktionen typischerweise die vier zentralen Eigenschaften Atomicity, Consistency, Isolation, Durability (ACID) gefordert (Saake, Heuer und Sattler 2018, Kap. 13.1.1):

Atomicity Die Atomarität stellt sicher, dass eine Transaktion entweder vollständig oder gar nicht ausgeführt wird. Tritt während der Ausführung ein Fehler auf, werden sämtliche bis dahin vorgenommenen Änderungen verworfen, sodass keine partiellen Zwischenzustände bestehen bleiben.

Consistency Die Konsistenz beschreibt die Eigenschaft, dass eine erfolgreich abgeschlossene Transaktion die Daten von einem gültigen Zustand in einen anderen gültigen Zustand überführt. Dabei müssen definierte Integritätsbedingungen erhalten bleiben. Kleppmann (2017, Kap. 7 Abschnitt „Consistency“) weist darauf hin, dass die Einhaltung solcher fachlichen Invarianten in der Verantwortung der Anwendung liegt und nicht wie die anderen Eigenschaften durch das Datenbanksystem garantiert werden kann.

Isolation Die Isolation gewährleistet, dass parallel ausgeführte Transaktionen sich nicht gegenseitig beeinflussen. Aus Sicht einer Transaktion soll die Ausführung so erscheinen, als würde sie allein auf dem Datenbestand arbeiten. Dadurch können Race Conditions und andere Nebenläufigkeitsprobleme vermieden werden.

Durability Die Dauerhaftigkeit garantiert, dass die Ergebnisse einer erfolgreich abgeschlossenen Transaktion auch nach Systemabstürzen oder anderen Fehlern dauerhaft erhalten bleiben.

Im Kontext von Background-Job-Processing sind insbesondere die Eigenschaften der Atomarität und Isolation von Bedeutung. Während die Atomarität verhindert, dass Statusänderungen eines Jobs nur teilweise durchgeführt werden, bildet die Isolation die Grundlage für die koordinierte Verarbeitung gemeinsamer Datenbestände durch mehrere parallel arbeitende Worker-Instanzen. Die relevanten Konzepte zum Erreichen dieser Eigenschaften werden in den folgenden Abschnitten näher betrachtet.

2.2.2 Transaktionsgrenzen

Während ACID die theoretischen Garantien einer Transaktion definiert, bestimmt die Festlegung von Transaktionsgrenzen, welche konkreten Datenbankoperationen des softwareseitigen Kontrollflusses zu einer unteilbaren, logischen Einheit zusammengefasst werden. Die Festlegung dieser Grenzen steuert, wann eine Datenbanksitzung eine Transaktion initiiert und zu welchem Zeitpunkt die Änderungen final festgeschrieben (Commit) oder im Fehlerfall vollständig verworfen (Rollback) werden (Kudraß 2015, S.250).

Die Wahl geeigneter Transaktionsgrenzen wird zusätzlich dadurch erschwert, dass fachliche und technische Transaktionen nicht zwangsläufig identisch sind. Eine fachliche Transaktion beschreibt eine zusammenhängende Geschäftsoperation, deren Ergebnis aus Sicht der Anwendung als Einheit betrachtet wird. Eine technische Transaktion hingegen bezeichnet die konkrete Transaktion auf der Datenbankebene, welche die ACID-Eigenschaften für die von ihr gekapselten Datenbankoperationen gewährleistet. Während eine fachliche Operation mehrere technische Transaktionen umfassen kann, können innerhalb einer technischen Transaktion auch nur Teilaspekte einer fachlichen Operation verarbeitet werden. Werden die Grenzen falsch gesetzt, können verschiedene Parallelitätsanomalien auftreten (Saake, Heuer und Sattler 2018, Kap. 13.1.2). Ein verbreiteter Ansatz zur Vermeidung dieser Anomalien ist die Verwendung von Sperren, auf die in Kapitel 2.2.3 näher eingegangen wird. Wichtig ist an dieser Stelle, dass die gewählten Transaktionsgrenzen direkten Einfluss auf die Dauer von Sperren und somit auch auf parallel ausgeführte Transaktionen haben.

Eine Möglichkeit besteht darin, jede Datenbankoperation in einer eigenen Transaktion auszuführen. Dadurch werden Sperren nur für sehr kurze Zeit gehalten, was die Parallelität und den Durchsatz des Systems erhöht. Allerdings können zusammengehörige Änderungen nicht mehr atomar ausgeführt werden. Tritt zwischen mehreren aufeinanderfolgenden Operationen ein Fehler auf, können bereits festgeschriebene Änderungen nicht mehr gemeinsam zurückgesetzt werden. Die betroffenen Daten können dadurch in einen fachlich falschen Zustand überführt werden.

Als naiver Gegenentwurf können die Transaktionsgrenzen entsprechend der fachlichen Transaktion gesetzt werden. Diese Strategie kann jedoch zu langlaufenden Transaktionen führen, welche die Leistungsfähigkeit des Datenspeichers beeinträchtigt. Zum einen halten Transaktionen im Beispiel des 2-Phasen-Sperrprotokolls sämtliche während ihrer Laufzeit akquirierten Sperren bis zum endgültigen Commit oder Rollback aufrecht (Kleppmann 2017, Kap. 7 Abschnitt „Two-Phase Locking (2PL)“). Dadurch steigt die Wahrscheinlichkeit, dass konkurrierende Transaktionen aufeinander warten müssen. Zum anderen können langlaufende Transaktionen die Bereinigung alter Datenversionen verzögern, da

diese weiterhin für aktive Transaktionen sichtbar bleiben müssen (The PostgreSQL Global Development Group (Hrsg.) 2026, Kap. 24.1.2). Dies erhöht den Ressourcenbedarf des Datenbanksystems und kann dessen Leistungsfähigkeit beeinträchtigen.

Im Software-Entwurf erfordert das Setzen der Transaktionsgrenzen daher eine kontinuierliche Abwägung zwischen der Systemperformance und dem Schutz vor Datenanomalien. Weit gefasste Transaktionsgrenzen reduzieren das Risiko der Entstehung von Inkonsistenzen, blockieren jedoch Systemressourcen und Datensätze. Eng gefasste Grenzen erhöhen den Durchsatz, können jedoch das Risiko konkurrierender Zugriffe und daraus resultierender Race Conditions erhöhen.

2.2.3 Synchronisationsverfahren und Isolationsstufen

Zur Umsetzung der Isolationseigenschaft von Transaktionen müssen konkurrierende Zugriffe auf gemeinsame Daten koordiniert werden. Eine verbreitete Strategie zur Durchsetzung der Isolation ist die pessimistische Synchronisation. Sie basiert auf der Annahme, dass konkurrierende Zugriffe auftreten können und verhindert diese bereits im Vorfeld durch Sperrmechanismen. Bevor eine Transaktion einen Datensatz verändert, wird dieser für andere konkurrierende Transaktionen gesperrt. Der Vorteil pessimistischer Verfahren besteht darin, dass Konflikte bereits vor ihrer Entstehung verhindert werden. Dadurch eignen sie sich insbesondere für Szenarien mit häufigen konkurrierenden Zugriffen. Nachteilig wirken sich jedoch zusätzliche Wartezeiten aus, da Transaktionen auf die Freigabe benötigter Sperren warten müssen. Bei hoher Auslastung kann dies die Parallelität und den Gesamtdurchsatz eines Systems reduzieren. Außerdem entsteht die Gefahr von Deadlocks (Kudraß 2015, S. 253–257).

Alternative Ansätze sind nicht sperrende Verfahren, beispielsweise die optimistische Synchronisation oder das Zeitstempelverfahren. Anstatt Daten vor der Bearbeitung zu sperren, werden bei der optimistischen Synchronisation Transaktionen zunächst ausgeführt und erst beim Schreiben geprüft, ob ein Konflikt entstanden ist. Ist es zwischen zwei oder mehreren Transaktionen zum Konflikt gekommen, müssen einzelne Transaktionen zurückgesetzt und wiederholt werden. Bei dem Zeitstempelverfahren werden Zeitstempel für jede Transaktionen, sowie der Zeitpunkt des letzten Lese- und Schreibzugriffs für jedes Datenbankobjekt verwaltet. Anhand dieser Zeitstempel kann geprüft werden, ob zwei Transaktionen in Konflikt zueinander stehen und ob eine der Transaktionen zurückgesetzt werden muss. Nicht sperrende Verfahren vermeiden die durch Sperren verursachten Wartezeiten und ermöglichen dadurch eine hohe Parallelität. Treten jedoch Konflikte auf, müssen betroffene Operationen wiederholt werden. Sie eignen sich daher insbesondere für Systeme, in denen konkurrierende Änderungen selten vorkommen (Kudraß 2015, S. 257–259).

Welche Synchronisationsstrategie geeignet ist, hängt von den Anforderungen des jeweiligen Systems und der Häufigkeit erwarteter Konflikte ab. Während pessimistische Verfahren Konflikte präventiv verhindern, akzeptieren optimistische Verfahren mögliche Konflikte und behandeln diese nachträglich. Zusätzlich können Datenbanksysteme die strenge Isolationsforderung von ACID durch unterschiedliche Isolationsstufen abschwächen. Diese Stufen legen fest, in welchem Umfang Transaktionen die Änderungen parallel

laufender Transaktionen beobachten und welche Parallelitätsanomalien dadurch im System auftreten können (Kudraß 2015, S. 251). Je nach Anwendungskontext kann es sein, dass nicht alle dieser Anomalien auftreten können, beispielsweise wenn nur Leseoperationen ausgeführt werden, oder dass deren Auftreten in Kauf genommen werden kann, um eine bessere Performance zu erhalten. Höhere Isolationsstufen bieten stärkere Konsistenzgarantien, können jedoch die Parallelität des Systems verringern (Saake, Heuer und Sattler 2018, Kap. 13.1.3).

2.3 Fehlertoleranz und Wiederherstellung

Verteilte Softwaresysteme sind zwangsläufig mit einer Vielzahl potenzieller Fehlerquellen konfrontiert. Prozesse können unerwartet terminieren, Netzwerkverbindungen unterbrochen werden oder externe Dienste zeitweise nicht erreichbar sein (Newman 2026, Kap. 1 Abschnitt „The Three Golden Rules Of Distributed Systems“). Beispielsweise kann es bei einzelnen Teilausfällen dazu kommen, dass unklar bleibt, ob ein Verarbeitungsschritt erfolgreich abgeschlossen wurde oder erneut ausgeführt werden muss. Da sich solche Fehler in komplexen Systemlandschaften nicht vollständig vermeiden lassen, muss ihre Existenz als grundlegende Systemeigenschaft betrachtet werden (Newman 2026, Kap. 1 Abschnitt „Failure Is Inevitable“).

Die Zuverlässigkeit eines Systems ergibt sich daher nicht primär daraus, Fehler vollständig zu verhindern, sondern daraus, wie gut mit ihrem Auftreten umgegangen wird. Fehlertoleranz bezeichnet in diesem Zusammenhang die Fähigkeit eines Systems, seine wesentlichen Funktionen trotz auftretender Störungen weiterhin bereitzustellen. Darüber hinaus muss ein System in der Lage sein, nach einem Fehler einen konsistenten Zustand wiederherzustellen und die Verarbeitung kontrolliert fortzusetzen, beziehungsweise neuzustarten (Newman 2026, Kap. 1 „What Is Resilience?“). Die folgenden Abschnitte stellen die hierfür relevanten Konzepte und Mechanismen vor.

2.3.1 Fehlerklassifikation

Da Fehler in verteilten Systemen unvermeidbar sind, stellt sich nicht die Frage, ob sie auftreten, sondern wie mit ihnen umgegangen werden soll. Die Wirksamkeit von Fehlertoleranzmechanismen hängt dabei wesentlich davon ab, die Ursache und Eigenschaften eines Fehlers korrekt einzuordnen. Nicht jeder Fehler kann durch dieselben Maßnahmen behandelt werden. Während einige Fehler durch eine erneute Ausführung einer Operation behoben werden können, sind andere dauerhaft und erfordern eine Anpassung der Eingabedaten oder eine manuelle Intervention. Die Klassifikation von Fehlern bildet daher die Grundlage für Entscheidungen über Retry-Strategien, Fehlerbehandlung und Wiederherstellungsmechanismen (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Darüber hinaus können nicht alle denkbaren Fehler mit vertretbarem Aufwand behandelt werden. Während kurzzeitige Infrastrukturprobleme, Prozessabstürze oder Kommunikationsfehler häufig durch geeignete Architekturmechanismen abgefangen werden

können, liegen andere Ereignisse, etwa großflächige Naturkatastrophen oder der vollständige Ausfall eines Rechenzentrums, gegebenenfalls außerhalb des Fehlerannahmemodells einer Anwendung. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Entscheidung darüber, welche Fehlerarten berücksichtigt werden und welche Risiken akzeptiert werden müssen (Kleppmann 2017, Kap. 1 Abschnitt „Reliability“).

Eine grundlegende Einteilung erfolgt in transiente und permanente Fehler. Transiente Fehler treten zeitlich begrenzt auf und verschwinden nach kurzer Zeit wieder. Ursachen liegen häufig in der technischen Infrastruktur eines Systems, beispielsweise in vorübergehenden Netzwerkstörungen, kurzzeitigen Überlastungen von Diensten, nicht erreichbaren Datenbanken oder dem Neustart einzelner Systemkomponenten (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Permanente Fehler hingegen bestehen unabhängig von der Anzahl der Wiederholungsversuche fort. Ihre Ursache liegt häufig in der Anwendung selbst oder in den zu verarbeitenden Daten. Beispiele hierfür sind ungültige Eingabedaten oder fehlende Berechtigungen. Newman (2026, Kap. 3 Abschnitt „Should You Always Retry?“) verdeutlicht dies anhand von HTTP-Fehlercodes wie 404 Not Found oder 403 Forbidden. Die Abgrenzung zwischen beiden Fehlerarten ist insbesondere für die automatisierte Fehlerbehandlung relevant. Während transiente Fehler häufig eine erneute Ausführung rechtfertigen, sollten permanente Fehler möglichst früh erkannt und gesondert behandelt werden.

Eine besondere Rolle spielen zudem Timeout-Fehler. In verteilten Systemen werden Timeouts verwendet, um potenzielle Ausfälle zu erkennen. Dabei kann jedoch nicht eindeutig unterschieden werden, ob tatsächlich ein Fehler vorliegt oder die Antwort lediglich ungewöhnlich lange braucht. Ein Timeout signalisiert daher nur, dass eine erwartete Antwort nicht innerhalb eines definierten Zeitfensters eingetroffen ist und lässt somit keine Aussage über eine Ursache zu (Bass 2021, Kap. 17.2 Abschnitt „Failure in the Cloud“).

2.3.2 Retry-/Backoff-Strategien

Die in Kapitel 2.3.1 eingeführte Unterscheidung zwischen transienten und permanenten Fehlern bildet die Grundlage für den Einsatz von Wiederholungsmechanismen. Da transiente Fehler nur zeitweise die Verarbeitung behindern, kann in solchen Fällen eine erneute Ausführung einer fehlgeschlagenen Operation erfolgreich sein, obwohl der ursprüngliche Versuch fehlgeschlagen ist (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“). Retry-Strategien verfolgen damit also das Ziel, die Erfolgswahrscheinlichkeit einer Verarbeitung zu erhöhen, ohne dass ein manueller Eingriff erforderlich wird. Retries sollten ausschließlich für Fehler durchgeführt werden, deren Ursache außerhalb der eigentlichen Geschäftslogik liegt und deren Fortbestand nicht dauerhaft erwartet wird. Permanente Fehler sollten dagegen unmittelbar als endgültig fehlgeschlagen behandelt werden, da eine erneute Ausführung die Erfolgswahrscheinlichkeit nicht erhöht. Ein typisches Szenario, in dem Retries sinnvoll sind, ist das Verschicken von Nachrichten (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“; Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability considerations“).

Da die Ursache eines transienten Fehlers potenziell auch für eine gewisse Zeit nach dem Auftreten des Fehlers weiterhin besteht, werden Wiederholungen üblicherweise nicht

unmittelbar durchgeführt. Stattdessen wird zwischen den einzelnen Versuchen eine Wartezeit eingefügt. Dieses Vorgehen wird als Backoff bezeichnet und soll verhindern, dass wiederholte Fehlversuche zusätzliche Last auf bereits beeinträchtigte Systeme erzeugen. Bekommt eine Anfrage beispielsweise ein Timeout, weil der Server zu viele Anfragen erhält, würden direkte Retries den Server noch weiter mit Anfragen verstopfen. Eine einfache Variante besteht darin, zwischen allen Wiederholungsversuchen dieselbe Wartezeit zu verwenden. In verteilten Systemen wird jedoch häufig ein exponentieller Backoff eingesetzt, bei dem sich das Warteintervall nach jedem fehlgeschlagenen Versuch schrittweise erhöht. Dadurch erhält das betroffene System mehr Zeit, sich von einer temporären Störung zu erholen, da die Dichte der Wiederholungsversuche reduziert wird (Newman 2026, Kap. 3 Abschnitt „Delays Between Retries“).

Neben der zeitlichen Staffelung von Wiederholungsversuchen muss auch deren Anzahl begrenzt werden. Andernfalls besteht die Gefahr, dass ein fehlerhafter Job unendlich oft ausgeführt wird und dadurch dauerhaft Ressourcen belegt. In der Praxis wird daher eine maximale Anzahl von Wiederholungsversuchen definiert. Wird diese überschritten, gilt die Verarbeitung als endgültig fehlgeschlagen und der Job wird in einen entsprechenden Fehlerzustand überführt (Newman 2026, Kap. 3 Abschnitt „How Many Retries Are Appropriate?“). Abbildung 2.3 visualisiert ein mögliches Vorgehen, das die oben beschriebenen Prinzipien umsetzt.

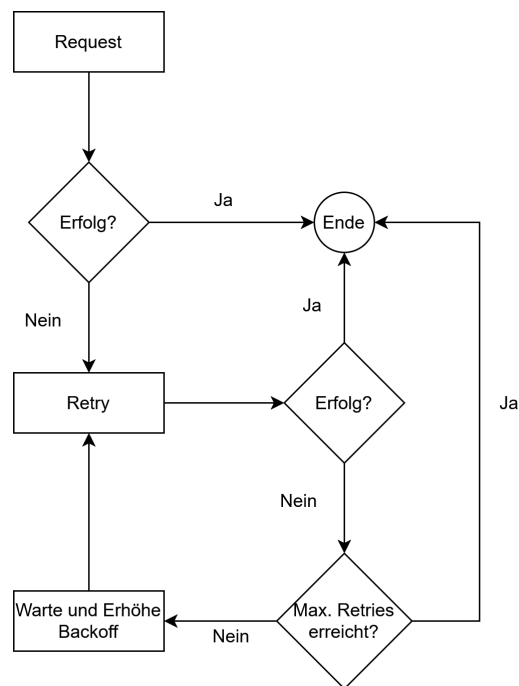


Abbildung 2.3: Verhalten bei transienten Fehlern

Retry- und Backoff-Strategien bilden damit einen wesentlichen Baustein zur Behandlung transients Fehler. Sie ermöglichen die automatische Wiederholung fehlgeschlagener

Verarbeitungen, ohne abhängige Systeme durch unmittelbar aufeinanderfolgende Wiederholungsversuche zusätzlich zu belasten.

2.3.3 Recovery

Neben transienten Fehlern, die häufig durch Retry-Mechanismen behandelt werden können, können auch Fehler auftreten, die den Ausführungsprozess einer Verarbeitung vollständig unterbrechen. Besondere Relevanz besitzen Ausfälle ausführender Komponenten während einer laufenden Bearbeitung. In einem solchen Szenario kann kein lokaler Retry mehr initiiert werden, da die ausführende Komponente selbst nicht mehr verfügbar ist. Recovery bezeichnet in diesem Kontext die Fähigkeit eines Systems, eine derart unterbrochene Verarbeitung zu erkennen und geeignete Maßnahmen zur Wiederaufnahme einzuleiten (Bass 2021, Kap. 4.2).

Voraussetzung für eine Wiederaufnahme ist zunächst die Erkennung unterbrochener Verarbeitungen. Wird eine Verarbeitung nach ihrer Übernahme durch eine ausführende Komponente unterbrochen, kann ein Zustand entstehen, in dem die Aufgabe aus Sicht des Systems weiterhin als aktiv gilt, obwohl tatsächlich keine Bearbeitung mehr stattfindet. Zur Erkennung solcher Situationen werden zusätzlich zu einem Statusmodell Heartbeat- oder Lease-/Timeout-Mechanismen benötigt (Bass 2021, Kap. 17.2 Abschnitt „Failure in the Cloud“). Bei Heartbeats sendet die ausführende Komponente in regelmäßigen Abständen Signale, anhand derer ihre Aktivität überwacht werden kann. Bleiben diese Signale über einen definierten Zeitraum aus, wird ein Ausfall vermutet und die Verarbeitung als potenziell unterbrochen betrachtet (Bass 2021, Kap. 4.2 Abschnitt „Detect Faults“). Lease- und Timeout-Mechanismen definieren lediglich eine Zeitspanne bzw. einen Zeitpunkt, bis zu dem eine Verarbeitung erlaubt ist. Beendet die ausführende Komponente die Verarbeitung nicht innerhalb dieses Rahmens, wird der Job als fehlgeschlagen interpretiert. Ist die Zeitspanne nicht passend konfiguriert, werden daher auch Jobs abgebrochen, die zu lange gebraucht haben, auch wenn bei deren Verarbeitung kein Fehler aufgetreten ist.

Nach der Erkennung eines Ausfalls muss die Verarbeitung erneut angestoßen werden. Hierfür wird die betroffene Aufgabe lediglich durch Zurücksetzen des Status wieder freigegeben. Die nächste Ausführungskomponente mit freien Ressourcen kann diesen Job wieder beanspruchen. Für die Wiederaufnahme existieren unterschiedliche Ansätze. Dabei wird zwischen flüchtigem und persistentem Zustand unterschieden. Während lokal im Arbeitsspeicher gehaltene Informationen beim Absturz eines Prozesses verloren gehen, kann persistent gespeicherter Zustand auch nach einem Ausfall weiterhin genutzt werden (Kleppmann 2017, Kap. 11 Abschnitt „Rebuilding state after a failure“). Die Verarbeitung auf Basis eines Zwischenzustands wieder aufzunehmen ist unter Umständen jedoch nicht trivial. Eine verbreitete Wiederherstellungsstrategie besteht daher darin, die für eine Verarbeitung relevanten Eingabedaten bis zum erfolgreichen Abschluss des Jobs zu speichern und eine unterbrochene Verarbeitung bei Bedarf auf Basis dieser Ursprungsdaten erneut auszuführen. Dies ist häufig einfacher und robuster als der Versuch, den exakten Laufzeitzustand eines abgestürzten Prozesses zu rekonstruieren und die Verarbeitung dort fortzusetzen (Bass 2021, Kap. 17.3). Welcher Ansatz gewählt wird, hängt

unter anderem von der Komplexität der Verarbeitung sowie vom Aufwand einer vollständigen Neuausführung ab. Unabhängig von Fortsetzung oder Neustart darf ein Ausfall nicht zu inkonsistenten Zuständen führen, die eine spätere Wiederaufnahme verhindern bzw. das Ergebnis beeinflussen. Daher muss sichergestellt werden, dass keine falschen oder unvollständigen Ergebnisse und Zustandsinformationen persistiert werden und dass die Speicherung immer atomar geschieht (Bass 2021, Kap. 4.2 Abschnitt „Recover from Faults“).

Die in diesem Kapitel beschriebene Wiederaufnahme führt zwangsläufig dazu, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden können. Da dabei gegebenenfalls nicht eindeutig festgestellt werden kann, ob eine vorherige Ausführung bereits teilweise oder vollständig erfolgreich war, müssen Systeme auf mögliche Mehrfachausführungen vorbereitet sein. Hieraus ergibt sich die Notwendigkeit idempotenter Verarbeitung, die im folgenden Abschnitt näher betrachtet wird.

2.3.4 Idempotenz

Die in den vorherigen Abschnitten beschriebenen Retry- und Wiederherstellungsmechanismen verfolgen eine At-Least-Once-Semantik und erhöhen dadurch die Zuverlässigkeit eines Systems, führen jedoch gleichzeitig dazu, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden können. Um trotz solcher Mehrfachausführungen konsistente Datenbestände sicherzustellen, wird das Konzept der Idempotenz eingesetzt. Eine Operation wird als idempotent bezeichnet, wenn ihre mehrmalige Ausführung denselben fachlichen Zustand erzeugt wie eine einmalige Ausführung (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Wiederholte Aufrufe verändern das Ergebnis somit nicht, wodurch Duplikate oder erneut ausgeführte Verarbeitungsschritte keine inkonsistenten Zustände verursachen. Wie in Kapitel 2.1.2 dargelegt, stellt Idempotenz damit eine wesentliche Voraussetzung dar, um auf Basis einer At-Least-Once-Semantik eine Exactly-Once-Semantik zu erreichen.

Idempotenz kann dabei auf unterschiedlichen Ebenen eines Systems umgesetzt werden. In Bezug auf Job-Processing muss bereits die Aufnahme eines Jobs in den Job-Store idempotent gestaltet werden, um zu verhindern, dass derselbe Job mehrfach im System erfasst wird. Ein verbreiteter Ansatz zur idempotenten Annahme von Anfragen ist die Implementierung eines idempotenten Receivers (Hohpe und Woolf 2005, Kap. 10 Abschnitt „Idempotent Receiver“) unter Verwendung eindeutiger Identifikatoren, wie beispielsweise eines Universally Unique Identifier (UUID). Bei einer möglichen Variante wird jeder Anfrage vom Sender eine eindeutige Kennung hinzugefügt, die vom Empfänger zusammen mit der Anfrage persistent gespeichert wird. Geht dieselbe Anfrage erneut ein, kann das System anhand dieser UUID erkennen, dass die Anfrage bereits verarbeitet wurde und eine redundante Speicherung verhindern (Newman 2026, Kap. 3 Abschnitt „Client Request IDs“; Richardson 2018, Kap. 3.3.6).

Darüber hinaus muss auch die eigentliche Verarbeitung eines Jobs so gestaltet werden, dass wiederholte Ausführungen und Wiederaufnahmen keine inkonsistenten Ergebnisse erzeugen (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Um eine Idempotente Verarbeitung sicherzustellen, darf ein Ergebnis nur einmal erfolgreich persistiert werden.

Außerdem muss das Speichern des Ergebnisses atomar ausgeführt werden. Datenänderungen müssen folglich über klar definierte Transaktionsgrenzen sowie ein Statusmodell abgesichert werden, um eine konsistente Weiterbearbeitung nach einem Crash zu gewährleisten (Kleppmann 2017, Kap. 7). Das Statusmodell garantiert, dass ausschließlich nicht fertige Jobs Ergebnisse speichern dürfen, während das Transaktionsmodell verhindert, dass nur Teile von Ergebnissen gespeichert werden.

Idempotenz bildet somit die Grundlage dafür, dass Retry- und Recovery-Mechanismen eingesetzt werden können, ohne die Konsistenz der verarbeiteten Daten zu gefährden. Sie stellt einen zentralen Baustein fehlertoleranter Background-Job-Processing-Systeme dar und ermöglicht die korrekte Verarbeitung trotz unvermeidbarer Mehrfachausführungen und Duplikate.

2.4 Stand der Forschung

Die Verarbeitung von Hintergrundaufgaben ist ein etabliertes Problemfeld verteilter Systeme. Entsprechend existieren zahlreiche Frameworks und Architekturmuster, die eine asynchrone Hintergrundaufführung von Aufgaben ermöglichen. Die Ansätze unterscheiden sich insbesondere hinsichtlich der verwendeten Infrastruktur, der Persistenzmechanismen sowie der Umsetzung von Fehlertoleranz und Wiederherstellung. Im Folgenden werden die für diese Arbeit relevanten Lösungsansätze vorgestellt und hinsichtlich ihrer Eigenschaften eingeordnet.

2.4.1 Brokerbasierte Ansätze

Eine weit verbreitete Klasse von Background-Job-Processing-Systemen basiert auf dedizierten Message Brokern wie RabbitMQ (Broadcom Inc. (Hrsg.) 2026a) oder Apache Kafka (Apache Software Foundation (Hrsg.) 2026). Hierbei werden Aufgaben in Form von Nachrichten an einen Broker übergeben, der die Entkopplung zwischen Produzenten und Konsumenten übernimmt. Worker beziehen die Nachrichten über den Broker und führen die zugehörige Verarbeitung aus. Der Broker läuft als eigenständige Middleware-Komponente entweder nativ oder virtuell als Server. Der Vorteil dieses Ansatzes liegt in der hohen Skalierbarkeit und der klaren Trennung zwischen Anwendungslogik und Nachrichteninfrastruktur. Darüber hinaus stellen moderne Broker Mechanismen zur Fehlertoleranz, Lastverteilung und Ausfallsicherheit bereit. Gleichzeitig entsteht durch brokerbasierte Architekturen zusätzliche betriebliche Komplexität, da neben der eigentlichen Anwendung weitere Infrastrukturkomponenten aufgesetzt, betrieben und überwacht werden müssen. Des Weiteren ergeben sich Herausforderungen bei der Konsistenz zwischen Anwendungsdatenbank und Message Broker. Problematisch ist beispielsweise die atomare Kopplung von Datenänderungen in der Datenbank und das Verschicken einer Nachricht. Da beide Systeme unabhängig voneinander arbeiten, müssen zusätzliche Mechanismen wie das Transactional Outbox Pattern eingesetzt werden, um Nachrichtenverlust oder Inkonsistenzen zu vermeiden (Richardson 2018, Kap. 3.3.7).

2.4.2 Datenbankbasierte Ansätze

Neben brokerbasierten Lösungen existieren Frameworks, die eine relationale Datenbank als zentralen Job-Store verwenden. Zu bekannten Vertretern dieses Ansatzes zählen unter anderem JobRunr (Rosoco BV (Hrsg.) 2026), Hangfire (Hangfire OÜ (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026b). Anstatt Aufgaben an einen separaten Broker zu übertragen, werden diese als persistente Datensätze in einer Datenbank gespeichert und von Workern direkt ausgelesen.

Die Nutzung einer relationalen Datenbank ermöglicht den Gebrauch der ACID-Eigenschaften, um das Speichern neuer Jobs sowie das Verwalten von Status- und fachlichen Datenänderungen konsistent innerhalb einer gemeinsamen Transaktion zu verarbeiten. In der Praxis verfolgen existierende Frameworks jedoch unterschiedliche Schwerpunkte und adressieren die Problemstellung jeweils aus einer anderen Perspektive. Hangfire garantiert nur eine At-Least-Once Ausführung und überlässt die Umsetzung von Idempotenz der Fachlogik. Während der Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) primär auf zeitgesteuertes Scheduling und clusteringbasierte Koordination fokussiert, stellt Spring Batch eine spezialisierte Infrastruktur für die blockweise Massenverarbeitung bereit, die auf periodische oder zeitgesteuerte Durchläufe ausgelegt ist. Modernere Java-Bibliotheken wie JobRunr und Hangfire integrieren zwar direkt ein automatisiertes Retry-Handling, kapseln die zugrundeliegenden Mechanismen jedoch als Blackbox, wodurch eine feingranulare Kontrolle und die präzise Analyse des Systemverhaltens bei Infrastrukturausfällen erschwert werden. Robuste und langlebige Workflow-Ausführungen wie Temporal (Temporal Technologies Inc. 2026) lösen die Frage nach Fehlertoleranz wiederum über eine dedizierte Engine, erzwingen damit jedoch einen eigenständigen Serverbetrieb und eine damit einhergehende infrastrukturelle Komplexität. Die konkrete Umsetzung der Nebenläufigkeitskontrolle, Wiederherstellung und Fehlerbehandlung ist stark von den jeweiligen Frameworks abhängig. Die zugrundeliegenden Prinzipien bleiben somit hinter der Schnittstelle der Frameworks verborgen und lassen sich nicht ohne Weiteres abstrahieren oder auf eigene Systementwürfe übertragen.

2.4.3 Einordnung und Forschungslücke

Die vorgestellten Ansätze zeigen, dass sowohl brokerbasierte als auch datenbankbasierte Architekturen zur Umsetzung von Background-Job-Processing-Systemen geeignet sind. Während bestehende Frameworks bereits umfangreiche Funktionalitäten für Scheduling, Retry-Mechanismen und Fehlertoleranz bereitstellen, liegt ihr Fokus primär auf der praktischen Nutzung, wodurch die zugrundeliegenden transaktionalen Mechanismen und deren Verhalten in kritischen Ausfallszenarien für den Entwickler oft intransparent bleiben. Insbesondere die Frage, unter welchen Voraussetzungen eine relationale Datenbank gleichzeitig als persistenter Job-Store und Koordinationsmechanismus eingesetzt werden kann, wird in der Literatur nur begrenzt behandelt. Konzepte wie Fehlertoleranz, Recovery, Idempotenz und Nebenläufigkeitskontrolle werden zwar häufig einzeln beschrieben, ihre Integration zu einer konsistenten Gesamtarchitektur bleibt jedoch oftmals implizit.

Die vorliegende Arbeit grenzt sich von bestehenden Frameworks ab, indem sie ein rein

datenbankgestütztes Pull-Modell entwirft, die Architekturentscheidungen begründet darlegt und das System bezüglich der angestrebten Fehlertoleranz evaluiert. Gleichzeitig wird durch die Kombination aus einem Statusmodell und der PostgreSQL-Funktionalität **FOR UPDATE SKIP LOCKED** untersucht, wie die Koordination direkt auf der Datenbankebene orchestriert werden kann, ohne einen separaten Message Broker einzusetzen und somit die Systemkomplexität niedrig zu halten.

3 Methodik

To be done

3.1 Literaturrecherche

To be done

3.2 Ergebnisgenerierung

To be done

Problemstellung

↓

Fehlermodell

↓

Anforderungen

↓

Zusicherungen

↓

Architektur

↓

Implementierung

↓

Evaluation

4 Anforderungen

Dieses Kapitel legt die Anforderungen an das im Rahmen der Arbeit entwickelte Background-Job-Processing-System fest und schafft damit die Grundlage für den späteren Architekturentwurf sowie dessen Evaluation. Zunächst wird der Systemkontext beschrieben, um die beteiligten Komponenten, Schnittstellen und Verantwortlichkeiten einzuordnen. Anschließend wird das betrachtete Fehlermodell abgegrenzt und damit festgelegt, welche Störungen und Randbedingungen für den Entwurf relevant sind. Darauf aufbauend werden präzise Systemzusicherungen formuliert, die das gewünschte Verhalten unabhängig von konkreten Implementierungsdetails beschreiben und als verbindlicher Maßstab für die Umsetzung und die Evaluation dienen.

4.1 Systemkontext

Im Rahmen dieser Arbeit wird ein datenbankgestütztes Background-Job-Processing-System entwickelt, dessen grundlegende Systemstruktur in Abbildung 4.1 dargestellt ist. Über eine REST-Schnittstelle können Aufträge durch einen Service oder ein Frontend erzeugt werden, die zunächst im Backend validiert und anschließend in einem relationalen Job-Store persistent gespeichert werden. Unabhängig vom ursprünglichen Aufruf übernehmen mehrere Worker-Prozesse anschließend die asynchrone Verarbeitung der gespeicherten Jobs. Zur Evaluation der entwickelten Architektur wird exemplarisch

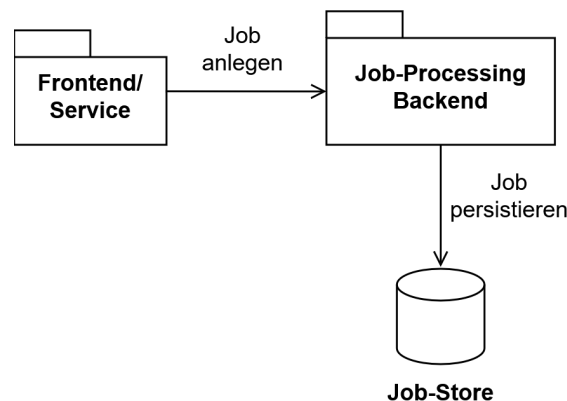


Abbildung 4.1: Systemkontext

ein Anwendungsszenario aus dem Umfeld der Verarbeitung technischer Wartungsdokumente betrachtet. Die konkrete fachliche Verarbeitung dient dabei ausschließlich als Demonstrations- und Evaluationsszenario, während der Schwerpunkt dieser Arbeit auf dem Entwurf der zugrunde liegenden Architektur für eine zuverlässige Hintergrundverarbeitung liegt. Ziel ist somit nicht die Entwicklung einer domänenspezifischen Fachanwendung, sondern einer allgemein einsetzbaren Architektur, deren Konzepte unabhängig vom gewählten Anwendungskontext auf andere Background-Job-Processing-Systeme

übertragbar sind.

Das betrachtete System vereint die wesentlichen Herausforderungen datenbankgestützter Background-Job-Processing-Systeme. Hierzu zählen insbesondere die zuverlässige und verlustfreie Persistierung eingehender Aufträge, die koordinierte Verarbeitung durch mehrere parallel arbeitende Worker sowie die Wiederaufnahme unterbrochener Verarbeitungen nach Teilausfällen. Die genau betrachtete Fehlermenge wird im folgenden Kapitel dargelegt.

4.2 Abgrenzung des betrachteten Fehlermodells

Verteilte Softwaresysteme können durch eine Vielzahl unterschiedlicher Fehler beeinträchtigt werden. Hierzu zählen unter anderem Hardwaredefekte, Softwarefehler, Netzwerkausfälle, Bedienfehler oder Cyberangriffe. Wie bereits in Kapitel 2.3 erläutert wurde, kann ein System jedoch nicht sämtliche denkbaren Fehler verhindern oder behandeln. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Abwägung, welche Fehlerarten berücksichtigt und welche Risiken akzeptiert werden. Diese Entscheidung orientiert sich zum einen am Anwendungskontext und den Anforderungen an das System und zum anderen an wirtschaftlichen Randbedingungen wie Entwicklungs- und Betriebsaufwand (Kleppmann 2017, Kap. 1 Abschnitt „Reliability“).

Die im Folgenden dargelegte Teilmenge betrachteter Fehler ist eine Auswahl der von Kleppmann (2017), Newman (2026) und Bass (2021) beschriebenen typisch vorkommenden Fehlern in verteilten Systemen. Ziel dieser Arbeit ist der Entwurf einer fehlertoleranten Architektur für ein datenbankgestütztes Background-Job-Processing-System. Betrachtet werden daher ausschließlich Fehlerklassen, die während des regulären Betriebs auftreten können und deren Auswirkungen durch Mechanismen der Anwendungsarchitektur beherrscht werden können. Fehler, deren Behandlung zusätzliche Infrastruktur oder organisatorische Maßnahmen wie Backups, Replikation oder physische Redundanz erfordern, liegen außerhalb des Betrachtungsumfangs dieser Arbeit.

Die Abgrenzung der betrachteten Fehler ist in Abbildung 4.2 zusammengefasst. Innerhalb des Systementwurfs werden im Wesentlichen vier Kategorien von Fehlern behandelt. An erster Stelle stehen Ausfälle einzelner Systemkomponenten sowie transiente Infrastruktur- und Kommunikationsstörungen. Hierzu zählen insbesondere das unerwartete Beenden eines Worker-Prozesses, Container-Neustarts, JVM-Abstürze, durch das Betriebssystem terminierte Prozesse, temporäre Netzwerkunterbrechungen oder kurzfristig nicht erreichbare Datenbanken. Dadurch kann es im System zu zwei Problemen kommen. Zum einen besteht die Gefahr, dass eine laufende Verarbeitung unterbrochen wird oder vorübergehend nicht fortgesetzt werden kann. Aufgrund der Charakteristik transienter Fehler wird dabei grundsätzlich davon ausgegangen, dass die betroffene Komponente oder Ressource nach kurzer Zeit wieder verfügbar ist. Zum anderen entsteht die Gefahr, dass eine Nachricht bzw. ein Job verloren geht. Durch diese Fehler gerät das Gesamtsystem in einen unklaren Zustand darüber, wie weit die Verarbeitung bereits fortgeschritten war bzw. ob der Job persistiert wurde. Da diese Störungen in verteilten Umgebungen regelmäßig auftreten können (Newman 2026, Kap. 1 Abschnitt „Failure Is Inevitable“),

muss die Systemarchitektur in der Lage sein, diese autonom zu überbrücken. Unter dem Aspekt der Fehlertoleranz sollen zur Behandlung transienter Fehler daher automatische Retries eingesetzt werden, um die Notwendigkeit von manuellem Eingreifen zu reduzieren.

Fehlerklasse	Relevant	Begründung
Komponenten und Infrastrukturausfälle	✓	Unterbrechen die Verarbeitung und gefährden die Zuverlässigkeit des Systems
Doppelte Requests	✓	Können zu mehrfach persistierten Jobs führen
Doppelte Ausführung	✓	Kann inkonsistente Seiteneffekte verursachen
Konkurrierender Zugriff	✓	Gefährdet die Konsistenz gemeinsamer Daten
Physischer Datenbankverlust	✗	Erfordert Backups & Redundanz
Totalausfall Infrastruktur	✗	Erfordern Disaster-Recovery Maßnahmen
Cyberangriffe	✗	Qualitätsmerkmal Sicherheit
Byzantinische Fehler	✗	Annahme: Vertrauensvolle Komponenten

Abbildung 4.2: Übersicht betrachteter Fehlerklassen

Als direkte Folge solcher Infrastrukturprobleme und der darauf reagierenden Wiederholungsmechanismen der Clients entstehen doppelte Requests. Ein Client sendet eine Anfrage erneut, weil eine Bestätigung aufgrund eines Timeouts ausblieb. Die Architektur soll im Rahmen der Zuverlässigkeit eine Exactly-Once-Semantik als Ziel haben und muss daher im beschriebenen Szenario durch Duplikaterkennung verhindern, dass identische Jobs mehrfach im Job-Store persistiert werden.

Eng damit verknüpft ist die doppelte Ausführung von Jobs. Bei der Wiederaufnahme eines zuvor abgebrochenen Jobs kann nach einem Fehler oft nicht eindeutig festgestellt werden, ob die vorherige Verarbeitung bereits Seiteneffekte erzielt hat. Die Architektur muss daher zwingend sicherstellen, dass wiederholte Ausführungen keine inkonsistenten Ergebnisse verursachen.

Des Weiteren werden konkurrierende Zugriffe betrachtet. Sobald Parallelverarbeitung mittels mehrerer, autonom agierender Worker eingesetzt wird, entsteht unweigerlich die Möglichkeit gleichzeitiger Zugriffe auf geteilte Datenstrukturen wie den zentralen Job-Store. Dies ist im klassischen Sinne zwar nicht direkt ein Fehler, jedoch können parallele Zugriffe ohne entsprechende Synchronisationsmechanismen potenziell zu Race Conditions und inkonsistenten Zuständen in der Persistenzschicht führen, weshalb deren Verhinderung wesentlicher Bestandteil des Entwurfs ist.

Dem gegenüber stehen Fehler, die bewusst außerhalb des Betrachtungsumfangs dieser Arbeit liegen. Ein physischer Datenbankverlust sowie ein Totalausfall der Infrastruktur können nicht durch die Softwarearchitektur des Job-Processing-Systems gelöst werden. Ein Verlust der Persistenzschicht oder ein Ausfall des gesamten Rechenzentrums infolge von großflächigen Stromausfällen oder Bränden am Serverstandort erfordert stattdessen infrastrukturelle Maßnahmen wie geografische Redundanz, kontinuierliche Backups und umfassende Disaster-Recovery-Konzepte (Newman 2026, Kap. 9).

Ebenso werden Cyberangriffe im Rahmen dieser Arbeit ausgeschlossen. Obwohl Angriffe wie SQL-Injections oder Denial-of-Service-Szenarien direkte Auswirkungen auf die Zuverlässigkeit eines Systems haben können, betreffen sie primär das Qualitätsmerkmal der Sicherheit. Ihre Abwehr erfordert spezialisierte Mechanismen wie strikte Eingabevalidierung, Ratenbegrenzungen, Authentifizierung und Angriffserkennungssysteme. Der Fokus dieser Arbeit liegt stattdessen auf der Gewährleistung von Fehlertoleranz bei regulärem Systembetrieb.

Schließlich werden auch byzantinische Fehler ausgeschlossen. Dies sind Fehler, bei denen kompromittierte oder defekte Komponenten willkürlich falsche oder manipulierte Informationen erzeugen und verbreiten (Kleppmann 2017, Kap. 8 Abschnitt „Byzantine Faults“). Die vorliegende Arbeit geht dahingehend von der Annahme ehrlicher Komponenten aus. Dies bedeutet, dass Komponenten zwar abstürzen oder vorübergehend nicht erreichbar sein können, im aktiven Zustand jedoch nie absichtlich falsche Informationen verbreiten.

Aus der gewählten Abgrenzung ergibt sich, dass der Schwerpunkt dieser Arbeit auf der Behandlung von Teilausfällen, konkurrierender Verarbeitung und transienten Infrastrukturfehlern liegt. Diese Fehlerszenarien besitzen unmittelbaren Einfluss auf die Zuverlässigkeit datenbankgestützter Background-Job-Processing-Systeme und bilden daher zusammen mit der Systembeschreibung aus Kapitel 4.1 die Grundlage für die im Folgenden formulierten Systemzusicherungen.

4.3 Zusicherungen

Die sich aus dem in Kapitel 4.1 beschriebenen Systemkontext und dem in Kapitel 4.2 definierten Fehlermodell ergebenden zentralen Anforderungen an das zu entwickelnde Background-Job-Processing-System sind in Abbildung 4.3 zusammengefasst. Auf Basis dieser Anforderungen lassen sich Eigenschaften ableiten, die das entwickelte Background-Job-Processing-System zu jedem Zeitpunkt gewährleisten muss, um die fehlerfreie Funktion des Systems bezogen auf die Zielsetzung der Arbeit sicherzustellen. Diese Eigenschaften werden im Folgenden als Systemzusicherungen bezeichnet. Im Gegensatz zu konkreten Architekturmechanismen beschreiben Systemzusicherungen ausschließlich das von der Architektur einzuhaltende Verhalten. Sie formulieren Invarianten des Systems, die unabhängig von der jeweiligen Implementierung oder dem aktuell betrachteten Fehlerszenario jederzeit gelten müssen. Die in Kapitel 5.1 entwickelten Architekturmechanismen dienen dazu, die Einhaltung dieser Zusicherungen sicherzustellen.

ID	Anforderung / Problemstellung
A-1	Erfassung eingehender Aufträge über die REST-Schnittstelle
A-2	Persistente Speicherung erfasster Jobs
A-3	Behandlung doppelter Client-Anfragen
A-4	Parallele Job-Verarbeitung durch mehrere Worker
A-5	Vermeidung von Race Conditions bei konkurrierendem Zugriff
A-6	Erkennung von Teilausfällen (Worker-Crashes, Container-Neustarts) und Wiederaufnahme/Wiederholung des betroffenen Verarbeitungsschrittes
A-7	Konsistente Verarbeitung trotz wiederholter Ausführung

Abbildung 4.3: Systemanforderungen

4.3.1 Joberstellung

Die Phase der Joberstellung umfasst die Entgegennahme eines Auftrags über die Schnittstelle sowie dessen dauerhafte Überführung in den zentralen Job-Store (A-1 & A-2). Unvollständige oder fehlerhafte Anfragen sollten dabei nicht zu inkonsistenten Zuständen in der Persistenzschicht führen, damit Worker-Prozesse sich darauf verlassen können, dass im Job-Store vorhandene Jobdaten fehlerfrei und vollständig verarbeitbar sind. Daraus leitet sich die erste Systemzusicherung ab:

Zusicherung Z-1 (Atomare Persistenz und Validität):

Das System garantiert, dass ein über die Schnittstelle eingehender Auftrag nur dann im Job-Store persistiert wird, wenn er die fachliche und syntaktische Validierung erfolgreich durchlaufen hat. Die Persistierung erfolgt atomar. Ein Auftrag ist für Worker-Prozesse entweder vollständig und in einem gültigen Initialzustand sichtbar oder existiert nicht.

Wie in Kapitel 4.2 dargelegt, führen transiente Kommunikationsstörungen (wie Netzwerk-Timeouts) auf Client-Seite zu Wiederholungsversuchen. Um zu verhindern, dass derselbe logische Auftrag infolge dieser Wiederholungen mehrfach erfasst und somit mehrfach ausgeführt wird (A-3), muss das System eine idempotente Auftragserfassung implementieren:

Zusicherung Z-2 (Duplikaterkennung bei der Auftragserfassung):

Das System garantiert, dass identische Anfragen, die von Aufrufern erneut gesendet werden, nicht zu mehrfach angelegten Jobs im Job-Store führen (*At-Most-Once-Semantik*).

4.3.2 Jobverarbeitung

Die Phase der Jobverarbeitung beschreibt den regulären Betrieb, in dem mehrere Worker-Prozesse autonom und parallel Aufträge aus dem Job-Store beziehen und ausführen. Aus

dem Zusammenspiel paralleler Instanzen (A-4) folgt die Konkurrenz beim Zugriff auf den gemeinsamen Job-Store, wodurch das Risiko von Race Conditions entsteht. Da im Recovery-Fall eine überschneidende Bearbeitung durch zwei Worker nicht ausgeschlossen werden kann, muss das System den Zugriff auf die gemeinsamen Daten koordinieren (A-5):

Zusicherung Z-3 (Exklusive Rechte):

Das System garantiert, dass niemals mehrere Worker-Instanzen gleichzeitig Zugriff auf den gleichen Datensatz haben. Insbesondere darf immer nur eine Worker-Instanz Schreibrechte auf einen bestimmten Job haben, sodass das Schreiben eines Ergebnisses nur durch den besitzenden Job möglich ist.

Die Fachlogik soll zudem klare Übergänge bieten. Im Rahmen der konsistenten Ergebniserzeugung (A-7) darf ein Job im System keine Teilergebnisse hinterlassen:

Zusicherung Z-4 (Atomare Wirkung von Ergebnissen):

Das System garantiert, dass das Eintragen eines fachlichen Ergebnisses (oder einer Fehlermeldung) und der dazugehörige Übergang in den Folgezustand eine atomare Einheit bilden. Nach außen sichtbar ist entweder das vollständige Ergebnis inklusive der Zustandsänderung oder keinerlei Auswirkung.

Zusätzlich müssen die Zustände der Jobverarbeitung einer klaren Richtung folgen, um eine mehrfache Ergebniserzeugung zu verhindern und die Konsistenz der Daten so zu schützen (A-7):

Zusicherung Z-5 (Finalität von Endzuständen):

Das System garantiert, dass der Lebenszyklus eines Jobs einer strikt gerichteten Zustandslogik folgt. Die Übergänge in die definierten Endzustände sind final und unumkehrbar. Kein Job kann nach Erreichen eines Endzustands erneut in einen aktiven Zustand versetzt werden.

4.3.3 Recovery

Das Unterkapitel Recovery adressiert das Verhalten des Systems bei Teilausfällen, wie dem unerwarteten Absturz von Worker-Prozessen oder abgebrochenen Verbindungen. Handelt es sich bei der Ursache der abgebrochenen Verbindung um einen transienten Fehler, soll dieser nicht unmittelbar zum endgültigen Fehlschlagen der Operation führen (A-6):

Zusicherung Z-6 (Automatische Wiederholungsversuche):

Das System kann transiente und permanente Fehler unterscheiden. Erkennt es einen transienten Fehler, behandelt es diesen durch automatische Wiederholungsversuche, ohne dass ein Nutzer eingreifen muss.

Das Gesamtsystem muss außerdem in der Lage sein, abgebrochene Verarbeitungen zu erkennen und den betroffenen Job ohne manuellen Eingriff wieder im System verfügbar zu machen (A-6):

Zusicherung Z-7 (At-Least-Once-Verarbeitungsgarantie):

Das System garantiert, dass ein einmal erfolgreich persistierter Job nicht verloren geht. Wird die Verarbeitung eines Jobs durch den Absturz einer Komponente unterbrochen, stellt das System sicher, dass der Job nach Erkennung des Ausfalls automatisch wieder zur Verarbeitung freigegeben und von einer verfügbaren Instanz erneut aufgenommen wird.

Da Wiederholungsversuche und Recovery-Mechanismen dazu führen können, dass die Bearbeitung eines Jobs mehrfach gestartet wird, muss sichergestellt werden, dass das fachliche Ergebnis eindeutig bleibt (A-7):

Zusicherung Z-8 (Eindeutigkeit des Ergebnisses und Idempotenz):

Das System garantiert, dass zu jedem im System erfassten Job exakt ein finales fachliches Ergebnis oder eine finale Fehlerbeschreibung gespeichert wird. Mehrfache oder abgebrochene Ausführungsversuche verfälschen das Endergebnis nicht und erzeugen keine duplizierten Ergebnisdatensätze.

Schließlich muss das System sicherstellen, dass Jobs nicht unendlich im System verbleiben oder in endlosen Fehlerschleifen gefangen sind:

Zusicherung Z-9 (Terminierung in endlicher Zeit):

Das System garantiert, dass jeder persistierte Job nach endlicher Zeit in einen der Endzustände übergeht. Kein Job verbleibt unendlich in einem aktiven bzw. ausstehenden Status.

5 Umsetzung

To be done

5.1 Architektur und Design

To be done (Einleitung)

5.1.1 Architekturentwurf

In Kapitel 2.1.3 wurde bereits dargelegt, dass das Queue-Based Load Leveling Pattern eine geeignete Option ist, die ausstehenden Jobs innerhalb eines Backend-Job-Processing-Systems zwischenspeichern. Eine verbreitete Variante, dieses Pattern für Nachrichten, oder wie im Fall dieser Arbeit für Jobs, umzusetzen, ist die Implementierung mithilfe dedizierter Message Broker (Kleppmann 2017, Kap. 11 Abschnitt „Message brokers“). Das im Rahmen dieser Arbeit entwickelte System nutzt hingegen bewusst eine relationale PostgreSQL-Datenbank als zentralen Job-Store. Durch die Nutzung PostgreSQL-spezifischer Mechanismen wie *SKIP LOCKED* in Kombination mit den vollständigen ACID-Transaktionsgarantien relationaler Datenbanken lässt sich bereits ein Teil der in Kapitel 4 erhobenen Anforderungen mittels der inhärenten Datenbankfunktionen realisieren. Die Datenbank übernimmt dabei nicht ausschließlich die Rolle eines persistenten Datenspeichers, sondern fungiert gleichzeitig als Warteschlange sowie als Synchronisationsmechanismus zwischen konkurrierenden Workern. Funktional orientiert sich die Architektur somit an den Prinzipien eines klassischen Message Brokers, nutzt jedoch die Konsistenzgarantien einer relationalen Datenbank als Grundlage für die Umsetzung der definierten Zusicherungen. Die Wahl eines datenbankbasierten Job-Stores stellt dabei einen bewussten Architektur-Trade-off dar. Im Vergleich zu In-Memory-basierten Messaging-Systemen entstehen durch den permanenten Zugriff auf einen externen persistenten Datenspeicher zusätzliche Latenzen und ein geringerer maximaler Nachrichtendurchsatz. Demgegenüber ermöglicht die relationale Datenbank jedoch eine dauerhafte Speicherung sämtlicher Jobs sowie atomare Zustandsübergänge und eine konsistente Wiederherstellung des Systemzustands nach Fehlern. Zusätzlich bietet eine relationale Datenbank mehr Sicherheiten vor verloren gegangenen Jobs, als typische In-Memory Message Broker, die dafür aufwändige Mechanismen benötigen. Die Haltung des Jobstatus in einer Datenbank hat außerdem für sehr rechenintensive Operationen den Vorteil, dass eine angefangene Operation nach einem Teilausfall sehr einfach fortgesetzt werden kann, anstatt den Job vollständig neu starten zu müssen, indem zur Wiederaufnahme der letzte bekannte Status des Jobs bei der Datenbank angefragt wird (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“). Gerade für kleine bis mittelgroße Systeme, in deren Anwendungskontext kein Jobverlust auftreten darf, stellt dies eine geeignete und praktikable Architektur dar, da auf eine aufwändige Clusterstruktur zur Absicherung von Jobverlusten verzichtet werden kann (Newman 2026, Kap. 8 Abschnitt „HowBrokers Work (In A Nutshell)“).

Die resultierende Gesamtarchitektur ist in Abbildung 5.1 dargestellt. Der Client bildet den Einstiegspunkt des Systems und erzeugt neue Jobs, die entsprechend A-1 über eine REST-Schnittstelle an das Backend übertragen werden. Im Rahmen des entwickelten Prototyps erfolgt dies lediglich direkt über einen Service, der die Kommunikation mit dem Backend-Application Programming Interface (API) kapselt. In der Praxis kann diese Komponente jedoch ebenso durch eine grafische Benutzeroberfläche oder ein beliebiges externes Anwendungssystem angesprochen werden. Die REST-API dient als Schnittstelle

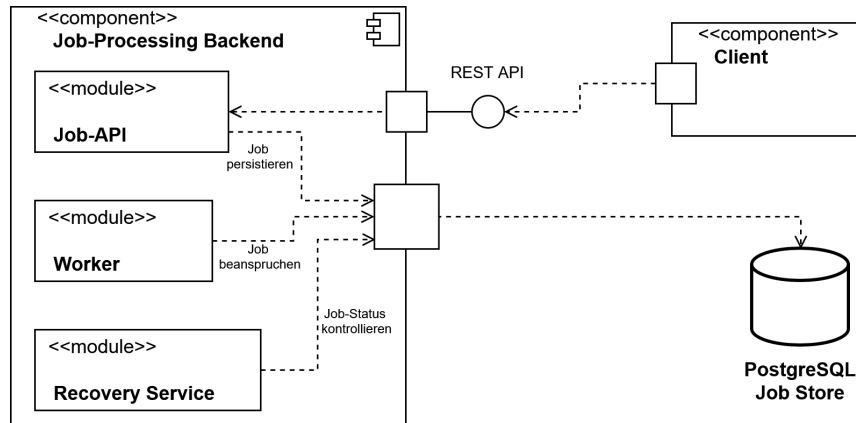


Abbildung 5.1: Abstrakte Architektur des entwickelten Systems

des Backends zur Erstellung neuer Jobs. Sie nimmt eingehende Anfragen entgegen, validiert diese (Z-1) und delegiert sie an die Servicelogik, die für die Persistierung neuer Jobs sowie die Verwaltung ihrer Zustände verantwortlich ist. Alle Jobs werden anschließend im zentralen Job-Store gespeichert (A-2), der neben den eigentlichen Jobdaten auch den aktuellen Bearbeitungszustand sowie gegebenenfalls erzeugte Ergebnisse enthält.

Die eigentliche Verarbeitung erfolgt durch mehrere parallel arbeitende Worker (A-4). Jeder Worker beansprucht einen Job exklusiv aus dem Job-Store, verarbeitet dessen Nutzdaten und speichert das erzeugte Ergebnis anschließend wieder in der Datenbank. Da sämtliche Worker auf denselben Job-Store zugreifen, kann die Verarbeitung durch Synchronisation auf Datenbankebene horizontal skaliert werden, ohne dass eine direkte Kommunikation zwischen den einzelnen Workern erforderlich ist.

Ergänzt wird die Architektur durch einen Recovery-Service, der den Job-Store periodisch auf verwaiste Jobs überprüft. Kann ein Worker einen bereits beanspruchten Job aufgrund eines Fehlers oder Absturzes nicht erfolgreich abschließen, erkennt der Recovery-Service anhand des abgelaufenen Leases, dass die Bearbeitung nicht abgeschlossen wurde. Der Status des betroffenen Jobs wird, sofern der Recovery-Service eine erneute Bearbeitung für sinnvoll erachtet, zurückgesetzt und steht damit einem beliebigen Worker erneut zur Bearbeitung zur Verfügung. Auf diese Weise wird verhindert, dass Jobs dauerhaft im Zustand **RUNNING** verbleiben (Z-9) oder infolge eines Worker-Ausfalls verloren gehen (Z-7).

5.1.2 Zustandsmodell

Zur Realisierung der in Kapitel 4 definierten Zusicherungen Z-3, Z-5, Z-7, Z-8 und Z-9 wird jeder Job durch ein explizites Zustandsmodell beschrieben. Der aktuelle Zustand eines Jobs wird dabei als Attribut des jeweiligen Datensatzes in der Datenbank gespeichert (vgl. Kapitel 5.1.3) und beschreibt den Fortschritt der Verarbeitung. Das Zustandsmodell bildet die Grundlage für sämtliche Entscheidungen bezüglich der Bearbeitung, Wiederherstellung und Beendigung eines Jobs.

Ein wesentlicher Vorteil dieses Ansatzes besteht darin, dass Datenbankzeilen nicht über den gesamten Verarbeitungszeitraum exklusiv gesperrt werden müssen. Sperren sind lediglich während kurzer atomarer Zustandsübergänge erforderlich, während die eigentliche Verarbeitung außerhalb einer Datenbanktransaktion erfolgt. Dadurch werden konkurrierende Zugriffe auf den Job-Store reduziert und Datenbankressourcen geschont, was sich positiv auf die Skalierbarkeit des Systems auswirkt.

Die in diesem System definierten Zustände sowie die zulässigen Zustandsübergänge sind in Abbildung 5.2 dargestellt. Nachdem ein neuer Job über die REST-Schnittstelle im

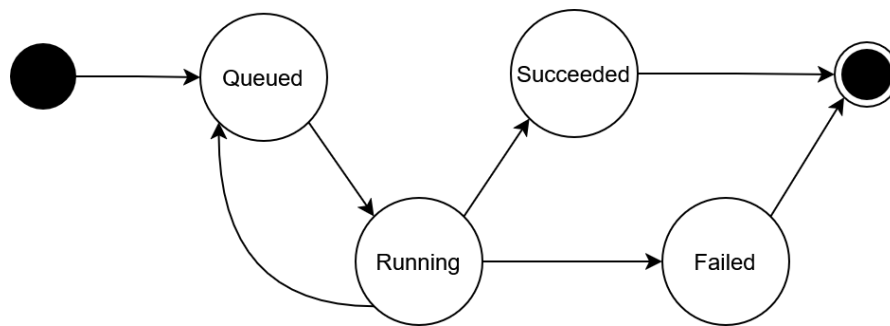


Abbildung 5.2: Zustandsmodell eines Jobs

Backend eingegangen ist, wird dieser persistent gespeichert und zunächst in den Zustand *Queued* versetzt. In diesem Zustand wartet der Job auf seine Bearbeitung und kann von einem Worker beansprucht werden. Beansprucht ein Worker den Job erfolgreich, erfolgt der Zustandsübergang nach *Running*. Dadurch wird sichergestellt, dass der Job während seiner Verarbeitung keinem weiteren Worker zur Verfügung steht.

Zur Verarbeitung existieren drei mögliche Folgezustände. Kann der Worker den Job erfolgreich bearbeiten und das erzeugte Ergebnis dauerhaft in der Datenbank speichern, wird der Job in den Endzustand *Succeeded* überführt. Tritt während der Verarbeitung hingegen ein Fehler auf oder beendet der Worker seine Arbeit nicht innerhalb der vorgesehenen Lease-Dauer, übernimmt der Recovery-Service die weitere Behandlung des Jobs. Ist gemäß der definierten Recovery-Strategie ein erneuter Ausführungsversuch zulässig, wird der Job zurück in den Zustand *Queued* versetzt und kann anschließend erneut von einem beliebigen Worker übernommen werden. Wurde dagegen die maximale Anzahl zulässiger Wiederholungsversuche erreicht oder liegt ein nicht behebbarer Fehler vor, wird der Job endgültig in den Endzustand *Failed* überführt. Jobs in diesem Zustand werden vom System nicht weiter automatisch verarbeitet und erfordern das manuelle Eingreifen

durch den Benutzer oder Administrator.

5.1.3 Entwurf der Datenbank

Die relationale Datenbank bildet den zentralen Job-Store des entwickelten Systems. Durch die Nutzung einer relationalen Datenbank mit ACID, wird automatisch die in Z-1 und Z-7 geforderte Dauerhaftigkeit angenommener Jobs garantiert. Sie übernimmt sowohl die Persistierung eingehender Jobs als auch die Verwaltung ihres aktuellen Bearbeitungszustands und gegebenenfalls erzeugter Verarbeitungsergebnisse. Das resultierende Datenbankschema ist in Abbildung 5.3 dargestellt. Hauptbestandteil des Schemas

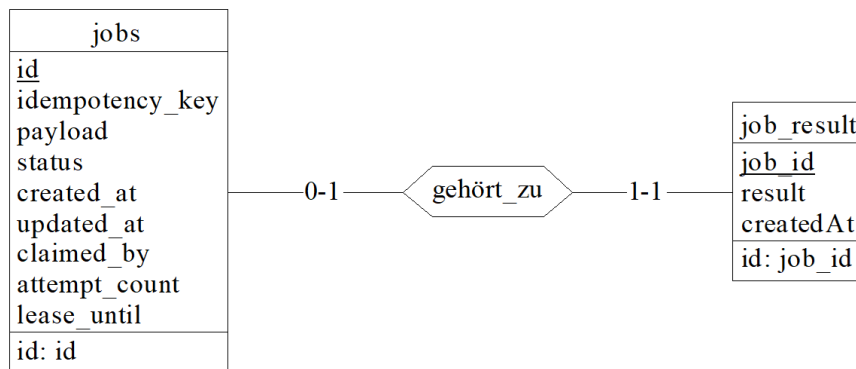


Abbildung 5.3: Datenbankschema des entwickelten Job-Processing-Systems

ist die Tabelle **jobs**. Jeder Job besitzt zunächst einen von der Datenbank erzeugten Primärschlüssel, der zur internen Identifikation dient. Zusätzlich wird für jeden Job ein vom Client bereitgestellter Idempotenzschlüssel gespeichert. Dieser stellt die in Z-2 geforderte Idempotenz in der Joberstellung sicher, auf die in Kapitel 5.1.4 näher eingegangen wird. Um das zu gewährleisten, müssen die Idempotenzschlüssel eindeutig sein. Daher ist die Spalte *idempotency_key* mit einem Unique-Constraint belegt.

Neben den Identifikationsmerkmalen wird der eigentliche Inhalt des Jobs als Payload persistent gespeichert. Im Beispielsystem wird dieses Feld als JSON-String modelliert. Abhängig von der Art des Jobs und den Anforderungen an das System kann dies jedoch auch in jeglicher anderen Form umgesetzt werden, sofern die nötigen Informationen für die Durchführung des Jobs enthalten sind. Durch die Persistierung der Payload bleibt die vollständige Anfrage dauerhaft verfügbar und kann im Fehlerfall erneut verarbeitet werden, ohne dass der Client den Job nochmals übertragen muss. Das bildet die Grundlage für Z-7 und Z-8.

Zur Umsetzung des in Kapitel 5.1.2 vorgestellten Zustandsmodells besitzt jeder Job ein Statusattribut, welches den aktuellen Bearbeitungszustand beschreibt. Ergänzend hierzu werden weitere Metadaten gespeichert, die für die Umsetzung der Fehlertoleranz erforderlich sind. Das Attribut **created_at** speichert den Zeitpunkt der Joberstellung. Indem Worker neue Jobs in der Reihenfolge ihres Erstellungszeitpunkts beanspruchen, wird

verhindert, dass ältere Jobs dauerhaft von neu eingehenden Jobs verdrängt werden und somit im Job-Store verbleiben. Dieses Verhalten adressiert Z-9. Während der Bearbeitung wird im Attribut `claimed_by` gespeichert, welcher Worker einen Job aktuell verarbeitet. Diese Zuordnung stellt sicher, dass ausschließlich der Worker, der einen Job erfolgreich beansprucht hat, dessen Verarbeitung abschließen und das Ergebnis persistent speichern darf. Dadurch wird die in Z-8 geforderte idempotente Ergebnisspeicherung unterstützt und verhindert, dass konkurrierende Worker im Recovery-Fall denselben Job gleichzeitig abschließen.

Für die Recovery-Strategie wird zusätzlich die Anzahl bisheriger Bearbeitungsversuche im Attribut `attempt_count` gespeichert. Dadurch kann die Anzahl automatischer Wiederholungsversuche begrenzt werden, sodass fehlerhafte Jobs nicht unbegrenzt erneut ausgeführt werden. Das Attribut `lease_until` dient der Umsetzung zeitbasierter Leases. Es gibt an, bis zu welchem Zeitpunkt ein Worker den beanspruchten Job exklusiv bearbeiten darf. Überschreitet die Bearbeitung diese Zeitspanne, kann der Recovery-Service den Job als verwaist erkennen und ihn entsprechend der definierten Recovery-Strategie erneut zur Bearbeitung freigeben. Diese beiden Attribute werden für die Umsetzung von Z-7 und Z-9 benötigt.

Die Verarbeitungsergebnisse werden in einer separaten Tabelle `job_result` gespeichert. Zwischen einem Job und seinem Ergebnis besteht dabei eine optionale Eins-zu-Eins-Beziehung. Ein Job besitzt höchstens ein Ergebnis, da die idempotente Verarbeitung sicherstellt, dass erfolgreiche Jobs nur einmal abgeschlossen werden können. Gleichzeitig gehört jedes gespeicherte Ergebnis eindeutig zu genau einem Job. Solange ein Job noch nicht erfolgreich bearbeitet wurde, existiert dementsprechend kein zugehöriger Ergebniseintrag.

Das dargestellte Datenbankschema bildet das minimale Schema zur Umsetzung der in dieser Arbeit betrachteten Fehlertoleranzmechanismen. In realen Anwendungen kann der Job-Store beliebig um weitere fachliche Attribute oder zusätzliche Tabellen erweitert werden. Ebenso ist es möglich, eine bereits bestehende relationale Datenbank zur Umsetzung des Job-Processing-Systems zu verwenden, indem das vorgestellte Schema in die vorhandene Datenbank integriert wird.

5.1.4 Joberstellung

Die Annahme neuer Jobs gehört zur Erstellungsphase neuer Jobs und bildet somit den Einstiegspunkt in das entwickelte Job-Processing-System. Zur Umsetzung einer Exactly-Once Auslieferung, die aus den Zusicherungen Z-2 und Z-6 folgt, sind verschiedene Mechanismen nötig, die im Folgenden vorgestellt werden. Der Ablauf der Jobannahme ist in Abbildung 5.4 dargestellt. Zur Erstellung eines neuen Jobs sendet der Client eine Anfrage über die REST-Schnittstelle an das Backend. Die Anfrage besteht aus zwei Bestandteilen: der eigentlichen Nutzlast (*Payload*), welche die durchzuführende Arbeit beschreibt, sowie einem vom Client erzeugten Idempotenzschlüssel. Dieser Schlüssel identifiziert einen Job eindeutig und ermöglicht es, mehrfach übertragene Anfragen desselben Jobs zuverlässig zu erkennen. Die Verwendung des Idempotenzschlüssels ist insbesondere für den Fall relevant, dass der Client eine Anfrage erneut sendet, obwohl der ursprüngliche Job

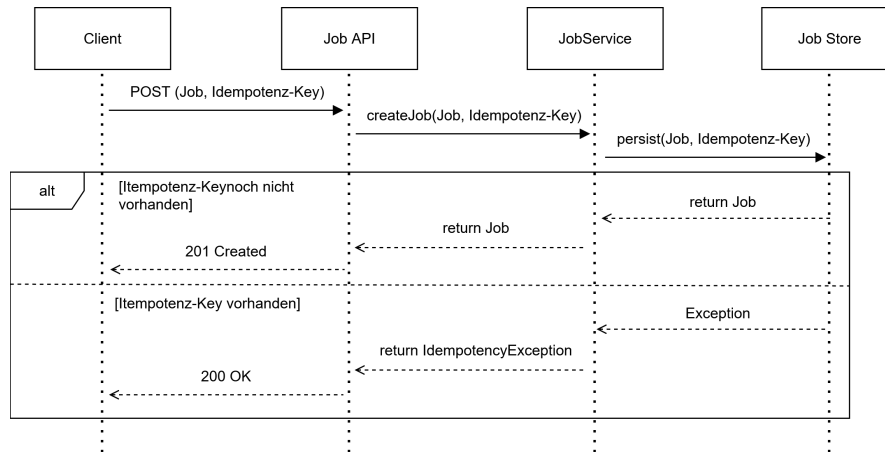


Abbildung 5.4: Ablauf der Jobannahme

bereits erfolgreich gespeichert wurde. Ein solches Szenario kann beispielsweise auftreten, wenn das Acknowledgement des Backends bei der Übertragung verloren geht und der Client deshalb fälschlicherweise von einer fehlgeschlagenen Joberstellung ausgeht. Durch den Idempotenzschlüssel kann das Backend erkennen, dass der betroffene Job bereits existiert und eine doppelte Persistierung verhindern. Damit wird Zusicherung Z-2 umgesetzt.

Nach Eingang der Anfrage leitet die REST-API diese an den Job-Service weiter. Dieser versucht, den Job zusammen mit dem Idempotenzschlüssel zu speichern. Existiert der Idempotenzschlüssel noch nicht, liefert die Anfrage den Job als Ergebnis. Der Service gibt den erstellten Job an die REST-API zurück, welche dem Client mit dem HTTP-Statuscode **201 Created** als Bestätigung der erfolgreichen Joberstellung antwortet.

Existiert dagegen bereits ein Job mit demselben Idempotenzschlüssel, wird ein Fehler aufgrund des Unique-Constraints der Datenbank ausgelöst. Der Service wirft daraufhin einen Fehler, der von der REST-API abgefangen wird. Da sich der gewünschte Job bereits im System befindet und somit der gewünschte Zielzustand erreicht wurde, beantwortet die API die Anfrage mit dem HTTP-Statuscode **200 OK**.

Kann während der Persistierung keine Verbindung zur Datenbank hergestellt werden oder tritt ein anderer transienter Fehler auf, wird der Persistierungsvorgang automatisch mehrfach mit exponentiellem Backoff wiederholt. Weitere Details zu Client- und Serverseitigen Retries werden in Kapitel 5.1.6 behandelt. Erst wenn der Job erfolgreich gespeichert wurde oder bereits existiert, wird dem Client ein erfolgreicher HTTP-Statuscode zurückgegeben. Kann der Job dagegen auch nach Ausschöpfen aller Wiederholungsversuche nicht gespeichert werden, antwortet das Backend mit dem HTTP-Statuscode **500 Internal Server Error**. Dadurch kann der Client erkennen, dass kein erfolgreicher Abschluss der Operation garantiert werden konnte und den Erstellungsversuch seinerseits gemäß der definierten Retry-Strategie wiederholen. Dieses Zusammenspiel zwischen client- und serverseitigen Wiederholungsmechanismen stellt sicher, dass die Zusicherung Z-6 eingehalten wird und somit kein direktes manuelles Eingreifen nach dem ersten fehlge-

schlagenen Versuch notwendig ist.

5.1.5 Exklusive Verarbeitung und Transaktionsmodell

Eine naheliegende Möglichkeit zur Absicherung der Hintergrundverarbeitung vor konkurrierenden Zugriffen besteht darin, eine Datenbanktransaktion über den gesamten Lebenszyklus eines Jobs geöffnet zu halten. Die Transaktion würde unmittelbar nach dem Beanspruchen eines Jobs beginnen und erst nach Abschluss der vollständigen Fachlogik sowie der Persistierung des Ergebnisses beendet werden. Wie bereits in Kapitel 2.2.2 erläutert, bringen lange Transaktionen einige Nachteile mit sich und werden daher auch für das in dieser Arbeit entworfene System nicht verwendet. Hintergrundjobs kapseln typischerweise langlaufende Berechnungen oder E/A-intensive Operationen (Microsoft Corporation (Hrsg.) 2026a). Während der gesamten Bearbeitungsdauer würden dadurch Datenbankverbindungen sowie Sperren unnötig lange gehalten werden müssen. Transaktionen sollten so lang wie nötig und so kurz wie möglich entworfen werden.

Aus diesem Grund verfolgt das entwickelte System einen anderen Ansatz. Die eigentliche Fachlogik wird vollständig außerhalb einer Datenbanktransaktion ausgeführt. Datenbanktransaktionen werden ausschließlich zur atomaren Synchronisation der Zustandsübergänge verwendet und deshalb möglichst kurz gehalten. Die Kombination aus dem in Kapitel 5.1.2 vorgestellten Zustandsmodell und kurzlaufenden Transaktionen ermöglicht eine exklusive Bearbeitung der Jobs, ohne Datenbankressourcen während der eigentlichen Verarbeitung zu blockieren.

Der Verarbeitungsablauf gliedert sich hierzu in drei Phasen, die in Abbildung 5.5 visualisiert sind. Zunächst beansprucht ein Worker innerhalb einer Datenbanktransaktion einen freien Job. Hierzu wird mittels *SELECT ... FOR UPDATE SKIP LOCKED* ein geeigneter Job ausgewählt und exklusiv gesperrt. Bereits von anderen Workern reservierte Jobs werden dadurch nicht blockierend abgewartet, sondern unmittelbar übersprungen. Zusätzlich wird der Jobstatus von *Queued* nach *Running* überführt und die Attribute für die Recovery wie *setClaimed_by*, *attempt_count* oder *lease_until* aktualisiert. Unmittelbar nach diesen Änderungen wird die Transaktion committet und die Datenbankverbindung wieder freigegeben. Anschließend erfolgt die eigentliche Verarbeitung des Jobs vollständig außerhalb einer Datenbanktransaktion. Da der Job bereits den Zustand *Running* besitzt, können andere Worker ihn währenddessen nicht erneut beanspruchen. Nach erfolgreichem Abschluss der eigentlichen Fachlogik darf ein Worker den Job nicht unmittelbar als erfolgreich markieren. Stattdessen wird zunächst eine neue Transaktion geöffnet, innerhalb der sowohl das Verarbeitungsergebnis persistent gespeichert als auch der Jobstatus von *Running* auf *Succeeded* aktualisiert wird. Beide Änderungen werden gemeinsam committet. Muss ein Job dagegen aufgrund eines Fehlers oder eines abgelaufenen Leases zurückgesetzt werden, werden innerhalb einer zusammengehörigen Transaktion der Status auf *Queued* zurückgesetzt sowie die Attribute *claimed_by* und *lease_until* zurückgesetzt. Dadurch ist sichergestellt, dass niemals ein Zustand entstehen kann, in dem zwar bereits ein Ergebnis gespeichert wurde, der Job jedoch weiterhin als *Running* oder *Queued* gekennzeichnet ist oder umgekehrt. Die gemeinsame Speicherung von Ergebnis und Jobstatus innerhalb derselben lokalen ACID-Transaktion gewährleistet

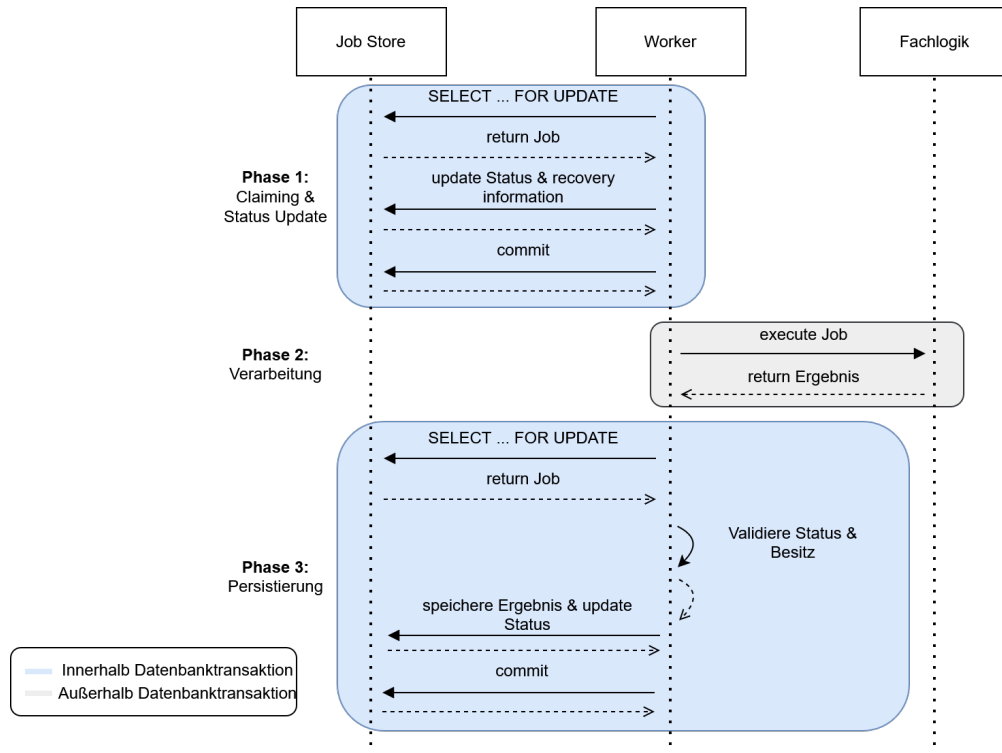


Abbildung 5.5: Transaktionsmodell

somit die in Z-4 geforderte Atomarität. Damit werden sämtliche Änderungen am Zustand eines Jobs atomar durchgeführt, während die eigentliche Fachlogik vollständig außerhalb der Transaktionsgrenzen verbleibt.

Für die Synchronisation konkurrierender Worker wird ein pessimistisches Sperrverfahren verwendet. Wie bereits in Kapitel 2.2.3 erläutert, eignen sich optimistische Synchronisationsverfahren insbesondere für Systeme, in denen konkurrierende Änderungen nur selten auftreten. Im entwickelten Job-Processing-System ist jedoch das Gegenteil der Fall. Da alle Worker neue Jobs entsprechend ihres Erstellungszeitpunkts aus derselben Warteschlange entnehmen, konkurrieren sie regelmäßig um dieselben Datensätze. Optimistische Verfahren würden in diesem Szenario zu einer hohen Anzahl abgebrochener Transaktionen und entsprechender Wiederholungsversuche führen. Kleppmann (2017, Kap. 7, Abschnitt „Pessimistic versus optimistic concurrency control“) weist darauf hin, dass sich optimistische Verfahren bei hoher Konkurrenz negativ auf den Gesamtdurchsatz auswirken, da die zusätzlich notwendigen Wiederholungen die Datenbank weiter belasten. Aus diesem Grund wird im Rahmen dieser Arbeit ein pessimistisches Sperrverfahren verwendet, bei dem konkurrierende Zugriffe bereits während der Auswahl eines Jobs verhindert werden. Durch die Nutzung expliziter Sperren wird die exklusive Bearbeitung unabhängig vom verwendeten Isolation Level gewährleistet. Aus diesem Grund genügt das in PostgreSQL standardmäßig verwendete Isolation-Level READ COMMITTED, da

die notwendige Synchronisation durch die expliziten Sperren erfolgt und strengere Isolationsebenen für den beschriebenen Ablauf keinen zusätzlichen Nutzen bieten.

Die Nutzung derselben relationalen Datenbank als Job-Store und zur Speicherung der Verarbeitungsergebnisse bietet den großen Vorteil, dass sämtliche Zustandsänderungen sowie die Persistierung eines Verarbeitungsergebnisses innerhalb lokaler ACID-Transaktionen erfolgen. Die Koordination mehrerer unabhängiger Systeme und damit die Notwendigkeit verteilter Transaktionen entfallen vollständig.

5.1.6 Retry-Strategie

Auf Grundlage der in Kapitel 2.3.2 beschriebenen Prinzipien werden Wiederholungsmechanismen gezielt an denjenigen Stellen der Architektur eingesetzt, an denen temporäre Infrastrukturfehler auftreten können, ohne dass die eigentliche Fachlogik fehlerhaft ist. Ziel ist es im Rahmen von Z-6, kurzfristige Störungen durch Wiederholungsversuche zu kompensieren und dadurch unnötige Recovery-Vorgänge oder Benutzerinteraktionen zu vermeiden.

Hierzu unterscheidet die Architektur zwischen der Kommunikation des Clients mit dem Backend und den internen Datenbankzugriffen der Worker und der Job-API. Beide Kommunikationswege stellen voneinander unabhängige Fehlerquellen dar und können daher unabhängig voneinander von transienten Infrastrukturfehlern betroffen sein.

Auf Clientseite werden Wiederholungsversuche für die Kommunikation mit der REST-Schnittstelle des Backends vorgesehen. Kann eine Anfrage aufgrund einer temporären Netzwerkstörung oder einer kurzfristigen Nichtverfügbarkeit des Backends nicht erfolgreich abgeschlossen werden bzw. bleibt die Erfolgsbestätigung aus, wird die Anfrage automatisch erneut übertragen. Da die Joberstellung durch den in Kapitel 5.1.4 beschriebenen Idempotenzmechanismus abgesichert ist, kann eine erneute Übertragung derselben Anfrage nicht zu einer mehrfachen Persistierung eines Jobs führen. Die Kombination aus Retry und Idempotenz schafft somit eine Exactly-Once-Semantik für die Joberstellung unter der Voraussetzung, dass der Fehler tatsächlich transient ist und die Parameter für Retry und Backoff passend konfiguriert sind.

Auch innerhalb des Backends werden Wiederholungsversuche für Datenbankoperationen vorgesehen, deren Fehlschlagen auf transiente Infrastrukturfehler zurückzuführen ist, beispielsweise kurzzeitige Netzwerkunterbrechungen oder die vorübergehende Nichtverfügbarkeit der Datenbank. Kann eine Operation aufgrund eines solchen Fehlers nicht abgeschlossen werden, wird sie automatisch erneut ausgeführt. Dagegen werden fachliche Fehler während der eigentlichen Jobverarbeitung bewusst nicht wiederholt, da deren Ursache nicht durch einen erneuten Ausführungsversuch beseitigt werden kann.

Zur strukturellen Entkopplung der Wiederholungslogik wird diese in einem eigenständigen Retry-Teilsystem gekapselt. Wie in Abbildung 5.6 dargestellt, nutzen die jeweiligen Komponenten, wie beispielsweise der Client im Frontend zur Erstellung der Jobs, einen zentralen RetryExecutor, der die auszuführende Operation über die Methode *execute(...)* entgegennimmt und deren Ausführung entsprechend einer konfigurierbaren Retry-Policy steuert. Der RetryExecutor enthält dabei selbst keine anwendungsspezifischen Entscheidungen darüber, wann eine Wiederholung erfolgen soll. Stattdessen fängt er Exceptions

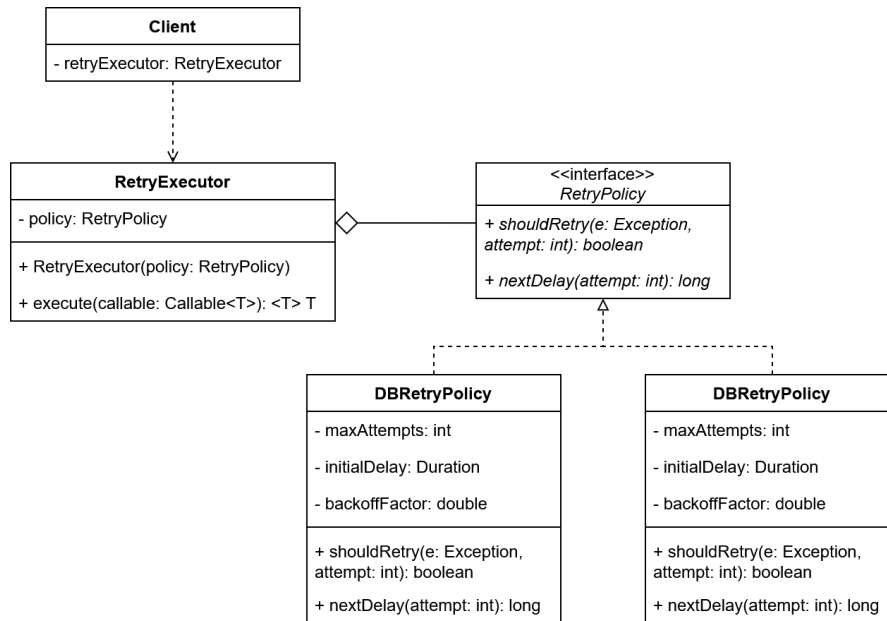


Abbildung 5.6: Klassendiagramm der Retry-Komponente

von der auszuführenden Operation und gibt sie an *shouldRetry(...)* weiter. Die spezifische Policy entscheidet dann, ob ein Retry sinnvoll ist oder nicht, indem sie die Exception klassifiziert und einen entsprechenden Wahrheitswert zurückliefert. Die Retry-Policies implementieren dabei eigene unterschiedliche Kriterien für Wiederholungsversuche, die sich in den behandelten Fehlerarten unterscheiden, sowie das zugehörige Backoff-Verhalten. Dadurch kann dasselbe Retry-Teilsystem sowohl für die Kommunikation zwischen Client und Backend als auch für interne Datenbankzugriffe der Worker verwendet werden, obwohl sich die jeweils behandelten Fehlerarten unterscheiden. Die Retry-Logik bleibt dadurch zentral gekapselt, während die konkrete Fehlerbehandlung an den jeweiligen Anwendungskontext angepasst werden kann. Die konkrete Implementierung des Retry-Executors sowie die Konfiguration der einzelnen Retry-Policies werden in Kapitel 5.2 erläutert.

5.1.7 Recovery

Die in Kapitel 5.1.6 beschriebene Retry-Strategie behandelt ausschließlich transiente Infrastrukturfehler, bei denen die ausführende Komponente weiterhin aktiv ist. Fällt ein Worker dagegen während der Verarbeitung vollständig aus oder bleibt dauerhaft hängen, kann kein lokaler Retry mehr erfolgen. Zur Behandlung solcher Fehler wird die Architektur daher um einen eigenständigen Recovery-Service ergänzt. Dieser überprüft den Job-Store in regelmäßigen Abständen auf potenziell verwaiste Jobs. Hierzu werden sämtliche Jobs betrachtet, die sich weiterhin im Zustand *Running* befinden, deren Lease jedoch bereits abgelaufen ist. Ein abgelaufenes Lease wird dabei als Indikator dafür verwendet,

dass die ursprüngliche Verarbeitung nicht innerhalb der vorgesehenen Zeit abgeschlossen werden konnte. Für jeden erkannten verwaisten Job entscheidet der Recovery-Service anschließend über das weitere Vorgehen. Wurde die maximal zulässige Anzahl an Ausführungsversuchen noch nicht erreicht, wird der Job nach dem in Kapitel 5.1.5 vorgestellten Transaktionsschema erneut zur Verarbeitung freigegeben. Anschließend kann der Job von einem beliebigen Worker erneut beansprucht werden. Wurde die konfigurierte Obergrenze der Wiederholungsversuche bereits erreicht, wird der Job stattdessen in den Endzustand *Failed* überführt und nicht erneut verarbeitet. Dadurch wird verhindert, dass dauerhaft fehlerhafte Jobs unbegrenzt Systemressourcen belegen und nie terminieren (Z-9).

Durch diesen Mechanismus können insbesondere Worker-Abstürze, JVM-Abstürze, Container-Neustarts sowie dauerhaft blockierte Worker in Bezug auf die Jobausführung behandelt werden (Z-7). Die eigentliche Terminierung oder der Neustart fehlerhafter Worker ist dagegen nicht Bestandteil des entwickelten Systems. Diese Aufgabe wird bewusst an eine externe Prozessüberwachung oder Orchestrierungsplattform, beispielsweise Kubernetes, delegiert. Das entwickelte System stellt lediglich sicher, dass durch den Ausfall eines Workers kein Job dauerhaft verloren geht.

Die Wiederaufnahme eines Jobs erfolgt in der entwickelten Architektur grundsätzlich nicht durch das Fortsetzen eines unterbrochenen Ausführungszustands, sondern durch eine vollständige Neuausführung auf Basis der ursprünglich persistierten Eingabedaten. Aus diesem Grund muss die Payload eines Jobs mindestens bis zum erfolgreichen Abschluss unverändert im Job-Store verbleiben. Dadurch kann jeder Wiederholungsversuch unter identischen Ausgangsbedingungen beginnen und unvollständig abgeschlossene Ausführungsversuche haben keine Auswirkungen auf den Erfolg oder das Ergebnis späterer Wiederholungen (Z-8).

Problematisch ist ein solches Vorgehen jedoch, wenn während der Verarbeitung externe Seiteneffekte entstehen. Wird beispielsweise wie in Abbildung 5.7 im Rahmen eines Jobs eine E-Mail versendet und der Worker stürzt anschließend vor dem erfolgreichen Abschluss der Verarbeitung ab, so wird der Job durch den Recovery-Service erneut zur Verarbeitung freigegeben. Die erneute Ausführung würde in diesem Fall dieselbe E-Mail ein weiteres Mal versenden. Die entwickelte Infrastruktur garantiert daher ausschließlich die idempotente Verarbeitung der internen Fachlogik des Job-Processing-Systems. Für externe Seiteneffekte kann eine Exactly-Once-Ausführung dagegen grundsätzlich nicht gewährleistet werden. Wie schon in Kapitel 2.1.2 erwähnt ist dies nur möglich, wenn das jeweils aufgerufene Zielsystem selbst eine idempotente Verarbeitung unterstützt, beispielsweise durch Nutzung von Idempotenzschlüsseln wie im Beispielsystem bei der Joberstellung (vgl. Kapitel 5.1.4). Unterstützt das Zielsystem keine entsprechende Semantik, kann eine Mehrfachausführung von Seiteneffekten infolge eines Recovery-Vorgangs nicht ausgeschlossen werden.

Ein Nebeneffekt dieser Architektur besteht darin, dass ein Job zeitweise von mehreren Workern gleichzeitig bearbeitet werden kann. Überschreitet beispielsweise ein Worker seine Lease-Dauer, wird der Job erneut freigegeben, obwohl der ursprüngliche Worker seine Verarbeitung möglicherweise noch beendet. Die beschriebene Recovery-Strategie versucht damit, die Erfolgswahrscheinlichkeit eines Jobs zu erhöhen. Sie verfolgt damit das Ziel einer At-least-Once-Ausführung von Jobs. Die daraus resultierenden Anforderungen an

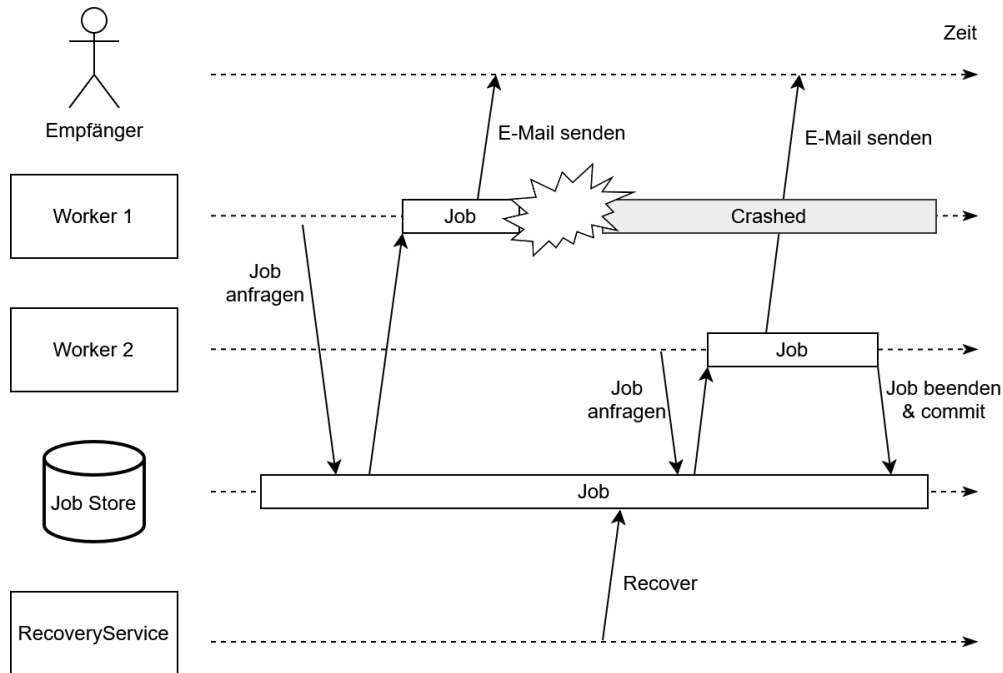


Abbildung 5.7: Mehrfachausführung mit Seiteneffekten

die idempotente Verarbeitung werden im folgenden Abschnitt betrachtet.

5.1.8 Idempotente Ergebnisgenerierung

Z-8 verlangt, dass pro Job maximal ein Ergebnis erzeugt wird. Das in Kapitel 5.1.5 vorgestellte Claiming-Verfahren stellt sicher, dass ein Job im Normalfall ausschließlich von einem einzelnen Worker bearbeitet wird. Durch den in Kapitel 5.1.7 genauer betrachteten Recovery-Fall kann diese Eigenschaft jedoch nicht immer garantiert werden. Daher muss sichergestellt werden, dass das fachliche Ergebnis nur einmal gespeichert werden kann oder die Fachlogik auch bei mehrfacher Ausführung in jedem Fall immer das gleiche Ergebnis erzeugt. Da der Jobstatus sowie die Jobergebnisse in derselben Datenbank verwaltet werden, bietet sich die Umsetzung der ersten Option mithilfe von Transaktionen an. Um die idempotente Verarbeitung auf diese Weise sicherzustellen, darf ein Worker ein Verarbeitungsergebnis ausschließlich dann persistent speichern, wenn er laut Job-Store weiterhin der Besitzer des Jobs ist. Vor der Speicherung wird daher der betroffene Datensatz gesperrt und innerhalb einer transaktional geschützten Operation erneut der aktuelle Jobstatus, die Worker-ID sowie die Gültigkeit des Leases überprüft. Nur wenn der Job sich weiterhin im Zustand *Running* befindet, die gespeicherte Worker-ID mit der des ausführenden Workers übereinstimmt und das Lease noch nicht abgelaufen ist, wird das Ergebnis übernommen und der Job erfolgreich abgeschlossen. Andernfalls wird der Speichervorgang verworfen, da bereits ein anderer Worker die weitere Verarbeitung übernommen hat oder der Job bereits abgeschlossen wurde. Dadurch kann jeder Job

höchstens einmal erfolgreich gespeichert werden, selbst wenn mehrere Worker denselben Job parallel ausgeführt haben. Das in Kapitel 5.1.7 bereits erwähnte Problem bezüglich Seiteneffekten in externen Systemen bleibt jedoch bestehen.

5.2 Implementierung

Die Implementierung orientiert sich an den im Abschnitt Kapitel 4.3 definierten Zusicherungen. Da einzelne Zusicherungen durch das Zusammenwirken mehrerer Komponenten und technischer Mechanismen realisiert werden, erfolgt die Darstellung der Implementierung nicht anhand der einzelnen Zusicherungen, sondern anhand der zentralen technischen Mechanismen des Systems. Die Zuordnung zu den jeweiligen Zusicherungen wird innerhalb der entsprechenden Abschnitte kenntlich gemacht.

5.2.1 Technologie-Stack

-JPA -> einfache Handhabung viel Automatisierung -> NativeQueries ermöglichen trotzdem viel Kontrolle beim Zugriff

5.2.2 Persistenz und Datenbankzugriff

Für den Zugriff auf die relationale Datenbank wird Spring Data JPA verwendet. Die Abbildung der persistenten Daten erfolgt durch die Entitätsklassen `Job` und `JobResult`, die jeweils mit der Annotation `@Entity` versehen sind.

Der Status eines Jobs wird als Java-Enumeration modelliert und über JPA persistent gespeichert. Damit wird sichergestellt, dass ausschließlich die im Zustandsmodell definierten Zustände verwendet werden können. Die Nutzdaten eines Jobs werden als JSON in der Datenbank persistiert. Dadurch kann die *Payload* eine Menge an strukturierten Informationen für die Jobspezifikation beinhalten. Alternativ könnte die *Payload* auch in Abhängigkeit von der Domain und dem Systementwurf als eigene Entität modelliert werden mit einer 1:1-Beziehung zum Job.

Eine besondere Bedeutung besitzt der *Idempotency-Key*, der zusätzlich zur technischen Identifikation eines Jobs gespeichert wird. Dieser Schlüssel wird vom Client als `UUID` erzeugt und dient zur Erkennung mehrfach übermittelter Anfragen. Auf Datenbankebene wird daher ein `UNIQUE`-Constraint für das entsprechende Attribut definiert. Dadurch kann die Datenbank selbst sicherstellen, dass die Idempotency-Keys eindeutig sind. In Kombination mit der Kennzeichnung *nullable=false* kann so die Eindeutigkeit erstellter Jobs auf Datenbankebene sichergestellt werden (Z-2).

Die Verarbeitungsergebnisse werden durch die Entitätsklasse `JobResult` repräsentiert. Umgesetzt wird die 1:1-Beziehung zwischen `JobResult` und `Job` über die `@OneToOne`-Annotation. Dabei wird die Fremdschlüsselbeziehung über `@JoinColumn` in der Tabelle des Ergebnisses umgesetzt. Die Beschränkung auf höchstens ein Ergebnis pro Job ist insbesondere für die idempotente Ergebnisspeicherung relevant (Z-8). Daher wird auch der Fremdschlüssel mit einem `UNIQUE`- als auch mit einem `NOT NULL`-Constraint belegt.

Der Datenbankzugriff wird über ein Repository-Interface gekapselt. Neben den von Spring Data JPA bereitgestellten Standardoperationen werden für das Job-Processing-System spezifische Abfragen definiert, die unmittelbar die Anforderungen an die konkurrierende Verarbeitung und die Recovery unterstützen. Für die Auswahl eines Jobs zur Verarbeitung wird beispielsweise die in listing 5.1 dargestellte native SQL-Abfrage definiert.

```
1 SELECT *
2 FROM jobs
3 WHERE status = :status
4 ORDER BY created_at
5 LIMIT :limit
6 FOR UPDATE SKIP LOCKED
```

Listing 5.1: Claiming eines Jobs im `JobRepository`

`FOR UPDATE` sperrt dabei den ausgewählten Datensatz für die laufende Transaktion zum Zwecke des exklusiven Zugriffs (Z-3), während `SKIP LOCKED` bereits durch andere Transaktionen gesperrte Datensätze ignoriert. Dadurch können mehrere Worker gleichzeitig nach verfügbaren Jobs suchen, ohne sich gegenseitig durch gesperrte Datensätze zu blockieren. Die Sortierung nach dem Erstellungszeitpunkt stellt dabei sicher, dass ältere Jobs bevorzugt verarbeitet werden (Z-9).

Für den Abschluss einer Verarbeitung und Recovery-Szenarien wird der in listing 5.2 gezeigte Befehl zum gezielten Zugriff auf einzelne Jobs bereitgestellt und gleichzeitig eine Sperre für diesen Datensatz gesetzt.

```
1 SELECT *
2 FROM jobs
3 WHERE id = :jobId
4 FOR UPDATE
```

Listing 5.2: Auswählen eines Jobs zur Bearbeitung

Auch dieser Befehl dient dazu, konkurrierende Änderungen an einem konkreten Job zu verhindern (Z-3).

Zusätzlich stellt das Repository eine Abfrage bereit, mit der Jobs anhand der Kombination ihres Status und des Ablaufs ihres Leases identifiziert werden können. Damit bildet die Abfrage aus listing 5.3 die Grundlage für die Erkennung verwaister Jobs durch den Recovery-Mechanismus.

```
1 SELECT j.id
2 FROM Job j
3 WHERE j.status = :status
4 AND j.lease_until < :timestamp
```

Listing 5.3: Finden verwaister Jobs

Wird der gesuchte Status auf *Running* und der `timestamp` auf den aktuellen Zeitpunkt gesetzt, liefert die Anfrage alle Jobs, die noch nicht beendet wurden und deren Lease gleichzeitig abgelaufen ist.

Die Transaktionskapselung wird durch `@Transactional` erreicht. Alle Datenbankoperationen in einer mit `@Transactional` gekennzeichneten Methode werden im Rahmen einer gemeinsamen Transaktion ausgeführt. Die konkrete Nutzung zur Umsetzung der in Kapitel 5.1 definierten Transaktionen wird bei den jeweiligen Verarbeitungsmechanismen in den folgenden Abschnitten erläutert.

Die implementierte Persistenzschicht stellt damit neben der dauerhaften Speicherung der Jobs und Ergebnisse (Z-7) auch die für die Fehlertoleranz erforderlichen Mechanismen zur Sicherstellung von Konsistenz (Z-1, Z-2, Z-4, Z-8) und zur Synchronisation konkurrierender Zugriffe (Z-3) bereit.

5.2.3 Joberstellung

Die Erstellung eines Jobs beginnt auf Clientseite mit der Zusammenstellung der für die Verarbeitung benötigten Nutzdaten. Diese werden als *Payload* in Form eines JSON-Dokuments repräsentiert. Zusätzlich erzeugt der Client einen UUID-basierten *Idempotency-Key*, der gemeinsam mit der Payload in dem Data Transfer Object (DTO) `CreateJobRequest` zusammengefasst wird.

Listing 5.4 zeigt, wie das Request-Objekt anschließend über den im Frontend implementierten `BackendRestClient` an die REST-Schnittstelle des Backends übertragen wird. Die Anfrage wird als HTTP-POST-Request an den Endpunkt `/job` gesendet und enthält das zuvor erzeugte Request-Objekt im HTTP-Body.

```
1 public ResponseEntity<CreateJobResponse> createJob(CreateJobRequest request) {  
2     return restClient.post()  
3         .uri("/job")  
4         .body(request)  
5         .retrieve()  
6         .toEntity(CreateJobResponse.class);  
7 }
```

Listing 5.4: POST eines Jobs

Im Backend wird die Anfrage durch den mit Spring Web implementierten `JobController` entgegengenommen. Der entsprechende Endpunkt ist über `@PostMapping("/job")` definiert und in listing 5.5 dargestellt. Der HTTP-Body wird durch `@RequestBody` automatisch in ein `CreateJobRequest`-Objekt überführt. Anschließend delegiert der Controller die eigentliche Joberstellung an den `JobService`. Die Validierung des Request-Objekts erfolgt dabei bereits auf Ebene der REST-Schnittstelle. Nach erfolgreicher Erstellung gibt der Controller den HTTP-Status 201 `Created` zurück. Tritt dagegen eine `IdempotencyException` auf, wird der bereits vorhandene Job mit 200 `OK` zurückgegeben. Andere Fehler werden als serverseitige Fehler mit 500 `Internal Server Error` beantwortet. Für den Client ist damit unabhängig davon, ob die ursprüngliche Anfrage oder eine erneute Übermittlung verarbeitet wurde, erkennbar, dass der angeforderte Job bereits im Job-Store vorhanden ist.

```
1 @PostMapping("/job")
```

```

2 public ResponseEntity<CreateJobResponse> createJob(@Valid @RequestBody
   CreateJobRequest request) {
3     try {
4         Job job = jobService.createJob(request);
5         return ResponseEntity
6             .status(HttpStatus.CREATED)
7             .body(new CreateJobResponse(job.getIdempotencyKey(),
8                 job.getStatus(), job.getCreatedAt()));
9     } catch (IdempotencyException e){
10        Job job = e.getExistingJob();
11        return ResponseEntity.status(HttpStatus.OK)
12            .body(new CreateJobResponse(job.getIdempotencyKey(),
13                job.getStatus(), job.getCreatedAt(),
14                "Job already exists"));
15    } catch (Exception e) {
16        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
17            .body(new CreateJobResponse(null, null, null, e.getMessage()));
18    }
19 }

```

Listing 5.5: POST Endpoint

Die eigentliche Erstellung und Persistierung des Jobs übernimmt der `JobService`. Hierzu wird zunächst über eine Factory aus dem Request-Objekt eine neue `Job`-Entität erzeugt. Anschließend versucht der Service, diese innerhalb einer Transaktion über das Repository zu persistieren. `@Transactional` stellt sicher, dass die Persistierung des Jobs atomar erfolgt, sodass im Fehlerfall keine inkonsistenten Daten in der Datenbank verbleiben.

```

1 @Transactional
2 public Job createJob(CreateJobRequest request) throws IdempotencyException {
3     Job job = JobFactory.createJob(request);
4     try{
5         repository.saveAndFlush(job);
6         return job;
7     } catch (DataIntegrityViolationException e){
8         if(!isConstraintViolation(e)){
9             throw e;
10        }
11        Optional<Job> existingJob = repository.findByIdempotencyKey(request.
getIdempotencyKey());
12        if (existingJob.isPresent()){
13            throw new IdempotencyException(existingJob.get());
14        }
15        throw e;
16    }
17 }

```

Listing 5.6: Transaktion zur Joberstellung

Eine Besonderheit ergibt sich aus der Eindeutigkeit des *Idempotency-Keys*. Wird versucht, einen Job mit einem bereits vorhandenen Schlüssel zu speichern, schlägt die Persis-

tierung aufgrund des entsprechenden **UNIQUE**-Constraints der Datenbank fehl. Die daraus resultierende **DataIntegrityViolationException** wird vom **JobService** auf eine Verletzung des für die Idempotenz relevanten Constraints geprüft. Handelt es sich um einen solchen Fall, wird der bereits vorhandene Job über seinen Idempotency-Key aus dem Repository geladen und eine **IdempotencyException** erzeugt, die den vorhandenen Job enthält.

Andere Integritätsverletzungen werden nicht als Idempotenzfall interpretiert und vom Service weitergereicht. Sie führen zusammen mit sonstigen unerwarteten Fehlern zur allgemeinen Fehlerbehandlung des Controllers und damit zu einer Antwort mit **500 Internal Server Error**.

5.2.4 Claiming und Verarbeitung

Die parallele Verarbeitung der Jobs wird durch den **WorkerManager** gesteuert, dessen Verhalten in listing 5.7 dargestellt ist. Dieser erzeugt beim Start der Anwendung einen Thread-Pool und übergibt jedem darin ausgeführten Thread eine eigene **Worker**-Instanz. Die Anzahl der parallel arbeitenden Worker wird über die Anwendungsconfiguration festgelegt und ist somit an den Systemkontext anpassbar. Jeder Worker erhält dabei Zugriff auf den **WorkerService** sowie auf eine Implementierung des **Processor**-Interfaces. Die Initialisierung des Thread-Pools und das Starten der Worker erfolgt daher nach Erzeugung des Spring-Kontexts über die mit **@PostConstruct** annotierte Methode *start()*, um zu garantieren, dass der **WorkerService** und der **Processor** bereits durch Spring injiziert wurden.

```
1 public WorkerManager(WorkerService workerService, Processor jobProcessor,
2                     @Value("${worker.count}") int workerCount) {
3     this.workerService = workerService;
4     this.jobProcessor = jobProcessor;
5     this.workerCount = workerCount;
6     this.executorService = Executors.newFixedThreadPool(workerCount);
7 }
8
9 @PostConstruct
10 void start() {
11     for (int i = 0; i < workerCount; i++) {
12         executorService.submit(new Worker(workerService, jobProcessor));
13     }
14 }
```

Listing 5.7: Initialisierung des Worker-Pools

Die eigentliche Verarbeitung erfolgt innerhalb der **Worker**-Instanzen. Jeder Worker besitzt eine eigene UUID, die bei seiner Erzeugung generiert wird. Diese wird beim Beanspruchen eines Jobs als *claimed_by* im Job-Store hinterlegt und ermöglicht dadurch die eindeutige Zuordnung eines laufenden Jobs zu demjenigen Worker, der ihn aktuell verarbeitet. Der **ExecutorService** ruft automatisch die in listing 5.8 dargestellte *run()*-Methode der übergebenen **Worker**-Instanz auf. Innerhalb der Methode wird für das Abarbeiten der

Jobs aus dem Job-Store gesorgt. Hierzu versucht der **Worker** zunächst, über die Methode *claimNextJob(UUID id)* des **WorkerService** einen verfügbaren Job zu beanspruchen. Wird kein Job gefunden, ist das **Optional**-Objekt leer. Der Worker wird daraufhin für eine kurze Zeit pausiert und führt anschließend eine erneute Abfrage durch. Beendet wird die Schleife erst, wenn der Thread unterbrochen wird.

```
1 @Override
2 public void run() {
3     while (!Thread.currentThread().isInterrupted()) {
4         try {
5             Optional<Job> job = workerService.claimNextJob(id);
6             if (job.isEmpty()) {
7                 if (!sleep(2000)) {
8                     break;
9                 }
10                continue;
11            }
12            JobResult result = processor.process(job.get());
13            workerService.finishJob(job.get().getId(), id, result);
14        } catch (Exception e) {
15            e.printStackTrace();
16        }
17    }
18 }
```

Listing 5.8: Ausführungsschleife eines Workers

Findet *claimNextJob* hingegen einen Job, sperrt die Methode diesen und gibt ihn an die **Worker**-Instanz zurück. Anschließend wird der Job über den **Processor** verarbeitet und das Ergebnis gespeichert.

Die Implementierung von *claimNextJob* ist in listing 5.9 veranschaulicht. Sie ist als transaktionale Operation implementiert und umfasst sowohl die Auswahl eines freien Jobs als auch dessen Überführung in den Zustand *Running*. Hierfür greift das Repository auf die in Kapitel 5.2.2 beschriebene Abfrage mit **FOR UPDATE SKIP LOCKED** zurück. Die Methode fordert dabei zunächst einen Job im Zustand *Queued* an. Existiert kein geeigneter Datensatz, wird ein leeres **Optional** zurückgegeben. Andernfalls wird der Datensatz durch das **JobRepository** gesperrt, die für die laufende Verarbeitung relevanten Attribute (vgl. Kapitel 5.1.5) werden innerhalb einer gemeinsamen Transaktion aktualisiert und der **Job** wird zurückgegeben. Besonders wichtig ist neben dem Status die Aktualisierung von *lease_until* für die Recovery, indem eine festgelegte Dauer auf den über **Instant** angefragten aktuellen Zeitpunkt addiert wird.

```
1 @Transactional
2 public Optional<Job> claimNextJob(UUID workerId) {
3     Optional<Job> result = jobRepository.findByStatus(JobStatus.QUEUED, 1);
4     if (result.isEmpty()) {
5         return Optional.empty();
6     }
7     Job job = result.get();
```

```

8     job.setStatus(JobStatus.RUNNING);
9     job.setClaimed_by(workerId);
10    job.setAttempt_count(job.getAttempt_count() + 1);
11    job.setLease_until(
12        Instant.now().plusSeconds(maximumJobDuration));
13    job.setUpdatedAt(Instant.now());
14
15    return Optional.of(job);
16 }

```

Listing 5.9: Beanspruchen eines Jobs

Durch die transaktionale Kapselung werden Auswahl und Statusänderung atomar durchgeführt. Nach Abschluss der Methode wird die Transaktion beendet und der Job befindet sich im Job-Store im Zustand *Running*. Die eigentliche Verarbeitung findet somit nicht innerhalb dieser Transaktion statt. Dadurch bleibt die Datenbankverbindung nur für die zur Reservierung erforderliche Zeit belegt.

Nach erfolgreichem Claiming übergibt der Worker den Job zur Verarbeitung an seine **Processor**-Instanz. Die eigentliche Fachlogik wird dadurch von der allgemeinen Worker-Infrastruktur getrennt. Das **Processor**-Interface definiert hierzu lediglich die Methode `process(Job)`, die einen **Job** entgegennimmt und ein **JobResult** zurückliefert. Der Worker selbst enthält damit keine jobspezifische Verarbeitungslogik. Stattdessen kann dieselbe Worker-Infrastruktur für unterschiedliche Arten von Hintergrundjobs verwendet werden, indem eine entsprechende **Processor**-Implementierung bereitgestellt wird. Im konkreten System übernimmt die Klasse **JobProcessor** diese Aufgabe. Die Verarbeitung eines Jobs findet vollständig außerhalb einer offenen Datenbanktransaktion statt. Der **Processor** kann daher auch länger laufende Berechnungen oder externe E/A-Operationen durchführen, ohne dabei eine Datenbankverbindung oder eine Sperre auf dem Job zu halten. Erzeugt die Fachlogik erfolgreich ein **JobResult**, wird dieses anschließend an den **WorkerService** zur Persistierung übergeben. Die dabei verwendete Transaktion sowie die Prüfung der Berechtigung des Workers zur Ergebnisübermittlung werden in Kapitel 5.2.5 behandelt.

Tritt während der eigentlichen Verarbeitung eine Exception auf, bricht die Verarbeitung ab und es wird kein Ergebnis gespeichert. Der Job verbleibt zunächst im Zustand *Running*. Die weitere Behandlung erfolgt in diesem Fall über den Recovery-Mechanismus.

5.2.5 Abschluss einer Jobverarbeitung

Nach erfolgreicher Ausführung der Fachlogik übergibt der Worker das erzeugte **JobResult** zusammen mit der Job-ID und seiner eigenen Worker-ID an den **WorkerService**. Die in listing 5.10 dargestellte Methode `finishJob(...)` übernimmt anschließend die atomare Persistierung des Verarbeitungsergebnisses sowie den Abschluss der Jobverarbeitung.

Wie bereits bei der Reservierung eines Jobs erfolgt auch dieser Schritt entsprechend dem Vorgehen aus Kapitel 5.1.5 atomar innerhalb einer Datenbanktransaktion mittels `@Transactional`.

```

1  @Transactional
2  public Job finishJob(UUID jobId, UUID workerId, JobResult jobResult)
3      throws JobNotFoundException {
4
5      Job job = jobRepository.findByIdForUpdate(jobId)
6          .orElseThrow(JobNotFoundException::new);
7
8      if (job.getStatus() != JobStatus.RUNNING)
9          throw new IllegalStateException();
10
11     if (!workerId.equals(job.getClaimed_by()))
12         throw new IllegalStateException();
13
14     if (job.getLease_until().isBefore(Instant.now()))
15         throw new LeaseExpiredException();
16
17     job.setStatus(JobStatus.SUCCESSFUL);
18     resultRepository.save(jobResult);
19     job.setResult(jobResult);
20     job.setUpdatedAt(Instant.now());
21
22     return job;
23 }

```

Listing 5.10: Abschluss einer Jobverarbeitung

Zu Beginn der Methode wird der betreffende Job über *findByIdForUpdate(...)* geladen. Die zugrunde liegende SQL-Abfrage verwendet **FOR UPDATE** und sperrt den Datensatz dadurch exklusiv bis zum Abschluss der Transaktion. Dadurch wird verhindert, dass Recovery-Prozesse oder andere konkurrierende Transaktionen den Job während seiner Finalisierung verändern können. Das Verhalten ist essenziell, um Z-8 garantieren zu können. Anschließend wird überprüft, ob sich der Job noch im Zustand *Running* befindet. Dadurch wird ausgeschlossen, dass bereits abgeschlossene oder zurückgesetzte Jobs erneut oder unberechtigt beendet werden. Danach erfolgt die Prüfung, ob die übergebene Worker-ID mit dem im Job gespeicherten Attribut *claimed_by* übereinstimmt. Nur der Worker, der den Job zuvor erfolgreich beansprucht hat, ist berechtigt, dessen Verarbeitung abzuschließen. Dieser Vergleich bildet die technische Umsetzung des im Architekturkapitel beschriebenen Fencing-Mechanismus. Hat der Job zwischenzeitlich einen anderen Besitzer erhalten, beispielsweise infolge eines Recovery-Vorgangs, wird die Ergebnisspeicherung abgebrochen. Zusätzlich wird geprüft, ob das zum Job gehörende Lease zum Zeitpunkt der Ergebnisspeicherung noch gültig ist. Ist das Lease bereits abgelaufen, wird eine **LeaseExpiredException** ausgelöst. Auch in diesem Fall wird kein Ergebnis persistiert, da nicht mehr sichergestellt werden kann, dass der Worker den Job weiterhin exklusiv besitzt. Die weitere Behandlung erfolgt anschließend durch den Recovery-Mechanismus. Erst nachdem sämtliche Prüfungen erfolgreich abgeschlossen wurden, erfolgt die eigentliche Persistierung des Verarbeitungsergebnisses. Hierzu wird zunächst der Jobstatus auf *SUCCESSFUL* gesetzt. Anschließend wird das **JobResult** gespeichert und dem entspre-

chenden Job zugeordnet. Schließlich wird noch der Zeitstempel der letzten Änderung aktualisiert. Da sämtliche Änderungen innerhalb derselben Datenbanktransaktion erfolgen, wird verhindert, dass ein Zustand entsteht, in dem zwar bereits ein Verarbeitungsergebnis existiert, der Job jedoch weiterhin als *Running* gekennzeichnet ist oder umgekehrt. Erst mit dem erfolgreichen Commit der Transaktion gilt die Verarbeitung eines Jobs als vollständig abgeschlossen. Schlägt eine der Konsistenzprüfungen fehl oder tritt während der Persistierung eine Exception auf, wird die laufende Transaktion automatisch zurückgerollt. Bereits durchgeführte Änderungen werden dadurch verworfen, sodass der Job-Store auch im Fehlerfall einen konsistenten Zustand beibehält (Z-4 & Z-8).

5.2.6 Retries

Die im Architekturkapitel vorgestellte Retry-Strategie wird durch ein eigenständiges Retry-Teilsystem umgesetzt, das unabhängig von den eigentlichen Fachkomponenten eingesetzt werden kann. Kern dieses Teilsystems ist die Klasse **RetryExecutor**, welche die allgemeine Steuerung von Wiederholungsversuchen übernimmt. Die konkrete Entscheidung, ob eine Operation nach einem Fehler erneut ausgeführt werden soll sowie die Berechnung der Wartezeit zwischen zwei Versuchen werden dagegen an eine konfigurierbare **RetryPolicy** delegiert. Dadurch bleibt die Retry-Logik zentral gekapselt, während unterschiedliche Komponenten jeweils eine auf ihren Anwendungsfall angepasste Policy verwenden können.

Der **RetryExecutor** besitzt lediglich eine öffentliche Methode **execute()**, der eine beliebige Operation in Form eines **Callable**-Objekts übergeben wird. Dadurch kann derselbe Mechanismus unabhängig davon verwendet werden, ob beispielsweise ein REST-Aufruf, ein Datenbankzugriff oder eine andere Operation wiederholt werden soll. Listing 5.11 zeigt den wesentlichen Aufbau der Implementierung.

```
1 public <T> T execute(Callable<T> callable) throws Exception {  
2     int attempt = 1;  
3     while (true) {  
4         try {  
5             return callable.call();  
6         } catch (Exception e) {  
7             if (!policy.shouldRetry(e, attempt)) {  
8                 throw e;  
9             }  
10            Thread.sleep(policy.nextDelay(attempt));  
11            attempt++;  
12        }  
13    }  
14 }
```

Listing 5.11: Generischer RetryExecutor

Die Methode führt die übergebene Operation zunächst unmittelbar aus. Tritt dabei keine Exception auf, wird das Ergebnis direkt an den Aufrufer zurückgegeben. Wird dagegen eine Exception ausgelöst, entscheidet die konfigurierte **RetryPolicy**, ob ein weiterer Aus-

führungsversuch sinnvoll ist. Hierzu erhält sie sowohl die aufgetretene Exception als auch die aktuelle Versuchsnummer. Lehnt die Policy einen weiteren Versuch ab, wird die Exception unverändert an den Aufrufer weitergereicht. Andernfalls berechnet die Policy zunächst die Dauer des nächsten Backoff-Intervalls und pausiert den Worker für diese Zeit, bevor die Operation erneut ausgeführt wird.

Die konkrete Retry-Strategie wird über das Interface `RetryPolicy` beschrieben. Dieses definiert die beiden Methoden `shouldRetry(...)` und `nextDelay(...)`. Während `shouldRetry(...)` entscheidet, ob ein Fehler erneut ausgeführt werden soll, bestimmt `nextDelay(...)` die Wartezeit bis zum nächsten Versuch. Dadurch werden Fehlerklassifikation und die Steuerung der Wartezeit vollständig vom eigentlichen Retry-Mechanismus getrennt.

Für die Berechnung des Backoff-Intervalls wird entsprechend dem Architekturentwurf ein exponentieller Backoff verwendet. Die Verzögerung ergibt sich aus

$$d_n = d_0 \cdot b^{n-1},$$

wobei d_n die Wartezeit des aktuellen Wiederholungsversuchs, d_0 die initiale Wartezeit, b den Backoff-Faktor und n die Nummer des aktuellen Wiederholungsversuchs bezeichnet. Mit jedem weiteren Fehlversuch vergrößert sich somit das Warteintervall exponentiell.

Für die unterschiedlichen Einsatzzwecke werden jeweils eigene Implementierungen der `RetryPolicy` verwendet. Listing 5.12 zeigt exemplarisch die Implementierung der für Datenbankzugriffe verwendeten Policy.

```
1  @Override
2  public boolean shouldRetry(Exception e, int attempt) {
3      if (attempt >= maxAttempts) {
4          return false;
5      }
6      return e instanceof DataAccessException;
7  }
8
9  @Override
10 public long nextDelay(int attempt) {
11     return (long) (initialDelay.toMillis()
12         * Math.pow(backoffFactor, attempt - 1));
13 }
```

Listing 5.12: Beispiel einer `RetryPolicy`

Die Datenbank-Policy klassifiziert ausschließlich Datenbankzugriffsfehler (`DataAccessException`) als potenziell transient und begrenzt gleichzeitig die maximale Anzahl der Wiederholungsversuche. Für die Kommunikation zwischen Client und Backend wird dagegen eine separate Policy eingesetzt, die insbesondere Netzwerkfehler, Verbindungsabbrüche sowie als temporär klassifizierte HTTP-Fehler berücksichtigt. Obwohl beide Policies unterschiedliche Fehlerarten behandeln, nutzen sie denselben generischen `RetryExecutor`.

Die Einbindung des Retry-Mechanismus erfolgt unmittelbar an den Stellen, an denen temporäre Infrastrukturfehler auftreten können. Die jeweilige Operation wird hierzu als

Lambda-Ausdruck an den `RetryExecutor` übergeben. Listing 5.13 zeigt dies exemplarisch für einen Datenbankzugriff im Backend.

```
1 Job job = retryExecutor.execute(  
2     () -> jobService.createJob(request)  
3 );
```

Listing 5.13: Nutzung des `RetryExecutors`

Wird Zeile 3 aus listing 5.6 mit diesem Ausdruck ersetzt und die Datenbank ist beispielsweise nicht erreichbar, wird der Aufruf von *creatJob(request)* automatisch erneut versucht.

Neben der Joberstellung wird derselbe Mechanismus ebenfalls beim Claiming neuer Jobs sowie beim Abschluss einer Verarbeitung eingesetzt. Im Frontend kapselt der `RetryExecutor` darüber hinaus sämtliche REST-Aufrufe an das Backend. Durch diese einheitliche Verwendung entsteht ein konsistentes Wiederholungsverhalten über alle Komponenten hinweg, ohne dass Retry-Logik mehrfach implementiert werden muss. Änderungen an Wiederholungsstrategien beschränken sich dadurch auf die jeweilige `RetryPolicy`, während die eigentlichen Fachkomponenten ausschließlich ihre fachliche Funktionalität implementieren.

5.2.7 Recovery

Die Wiederherstellung verwaister Jobs wird durch die beiden Komponenten `RecoveryService` und `RecoveryExecutor` umgesetzt. Während der `RecoveryService` die periodische Suche nach wiederherzustellenden Jobs übernimmt, implementiert der `RecoveryExecutor` die eigentliche Recovery-Logik innerhalb einer Datenbanktransaktion.

Der `RecoveryService` wird mithilfe der Spring-Annotation `@Scheduled` in festen Zeitabständen ausgeführt. Im entwickelten System erfolgt ein Recovery-Zyklus alle 30 Sekunden. Dabei werden zunächst sämtliche Jobs ermittelt, deren Lease bereits abgelaufen ist und deren Status weiterhin `RUNNING` lautet. Hierzu wird die bereits in Kapitel 5.2.2 vorgestellte Repository-Abfrage verwendet. Die Abfrage liefert ausschließlich die Identifikatoren der betroffenen Jobs zurück und verwendet bewusst keine Sperren. Dadurch werden während der Suche keine Datenbankzeilen unnötig blockiert und konkurrierende Worker können ihre Verarbeitung ohne zusätzliche Wartezeiten fortsetzen. Für jede gefundene Job-ID wird anschließend der `RecoveryExecutor` aufgerufen.

```
1 @Scheduled(fixedDelay = 30, timeUnit = TimeUnit.SECONDS)  
2 public void recoverCycle() {  
3     List<UUID> ids =  
4         repository.findByStatusAndLease_untilBefore(  
5             JobStatus.RUNNING, Instant.now());  
6  
7     for (UUID id : ids) {  
8         recoveryExecutor.recoverJob(id);  
9     }  
10 }
```

Listing 5.14: Periodischer Recovery-Zyklus

Die eigentliche Wiederherstellung erfolgt innerhalb der Methode `recoverJob()`, die mit `@Transactional` gekennzeichnet ist. Zu Beginn der Transaktion wird der betreffende Job erneut über seine ID geladen und dabei mittels `SELECT FOR UPDATE` exklusiv gesperrt. Die Sperre stellt sicher, dass Recovery und konkurrierende Worker den Zustand desselben Jobs nicht gleichzeitig verändern können.

Da zwischen der Suche nach abgelaufenen Leases und dem Beginn der Recovery-Transaktion bereits Änderungen am Job erfolgt sein können, wird dessen aktueller Zustand erneut überprüft. Insbesondere kann ein Worker den Job zwischenzeitlich noch erfolgreich abgeschlossen haben. Befindet sich der Job daher nicht mehr im Zustand `RUNNING`, wird die Recovery ohne weitere Änderungen beendet. Die erneute Prüfung verhindert, dass bereits erfolgreich abgeschlossene Jobs aufgrund einer zwischenzeitlichen Zustandsänderung fälschlicherweise erneut in die Warteschlange aufgenommen werden.

Anschließend wird anhand des `attempt_count` entschieden, ob der Job erneut ausgeführt werden darf. Wurde die konfigurierte maximale Anzahl an Ausführungsversuchen noch nicht erreicht, wird der Job wieder in den Zustand `QUEUED` überführt. Andernfalls wird er als `FAILED` markiert. Dadurch werden endlose Wiederholungszyklen dauerhaft fehlerhafter Jobs verhindert.

Unabhängig von der aktuellen Versuchsanzahl werden die Attribute `claimed_by` und `lease_until` zurückgesetzt sowie der Änderungszeitpunkt aktualisiert.

```
1 @Transactional
2 public void recoverJob(UUID id) {
3     Job job = repository.findByIdForUpdate(id).orElse(null);
4
5     if (job == null || job.getStatus() != JobStatus.RUNNING) {
6         return;
7     }
8     if (job.getAttempt_count() < maxAttempts) {
9         job.setStatus(JobStatus.QUEUED);
10    } else {
11        job.setStatus(JobStatus.FAILED);
12    }
13
14    job.setClaimed_by(null);
15    job.setLease_until(null);
16    job.setUpdatedAt(Instant.now());
17 }
```

Listing 5.15: Transaktionale Wiederherstellung eines Jobs

6 Evaluation

To be done

6.1 Fehlerszenarien

6.2 Durchführung

6.3 Ergebnisse

7 Diskussion

To be done

7.1 Gewonnene Erkenntnisse

7.2 Einschränkungen und Grenzen der Arbeit

Ausschließlich Notizen

Diese Arbeit betrachtet die Fehlertoleranz ab dem Zeitpunkt, zu dem ein Job persistent im Job-Store vorliegt; die zuverlässige Auftragserzeugung/Client-Kommunikation (z.B. Outbox, Messaging) ist nicht Gegenstand. Consumers may crash at any time, so it could happen that a broker delivers a message to a consumer but the consumer never processes it, or only partially processes it before crashing. In order to ensure that the message is not lost, message brokers use *acknowledgments*: a client must explicitly tell the broker when it has finished processing a message so that the broker can remove it from the queue." (Kleppmann 2017, Kap.11 -> Acknowledgments and redelivery bzw. Message brokers)

REST hat keinen Puffer und ist synchron -> Client und Server muss während der ganzen Kommunikation laufen, damit Kommunikation funktioniert. Weitere Nachteile in Referenc: (Richardson 2018, Kap.3.2.2)

Kein Fokus auf Kommunikation im allgemeinen, weder wie oben genannt die Erstellung von Jobs, als auch die Benachrichtigung bei Fertigstellung. Dafür bräuchte es Transaktionales messaging bspw. mittels transactional outbox pattern (Richardson 2018, Kap.3.3.7) oder asynchrones Messaging (Richardson 2018, Kap.3.4.2)

Es gibt keine perfekte reliability (Kleppmann 2017, Kap. 1, Abschnitt „Reliability“)

Load Leveling umgesetzt?

Performance von DB gegenüber Message Queues

Keine Fehlertoleranz in Bezug auf Datenbeständigkeit (z.B. Festplatte des Job-Stores geht kaputt), Naturkatastrophen (Ganzer Standort fällt aus z.B. wegen Überschwemmung) (Newman 2026, Kap. 9)

Es gibt keine Möglichkeit absolute Garantien zu geben, nur viele Möglichkeiten der Risikoreduktion (Kleppmann 2017, Kap. 7 Abschnitt „The Meaning of ACID“-> Ende)

um performance zu verbessern könnte man verschiedene checkpoints in die verarbeitung einbringen, sodass bei einem Abbrechen einer Task nicht die gesamte Task neu gestartet wird sondern ab einem bestimmten punkt wieder angefangen wird (Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability considerations“).

vorsicht bei dinge bei denen kein rollback gemacht werden kann: versendet ein worker eine email, stürzt ab und die operation wird wiederholt wird potenziell noch eine email versendet. das muss gesondert abgefangen werden

Es können Probleme entstehen, wenn zu viele Anfragen rein kommen, die die DB nicht gleichzeitig verarbeiten kann (vgl. Exmaple (Microsoft Corporation (Hrsg.) 2026b)) Dafür

bräuchte es separates Queue-Based Load Leveling für das Messaging an sich, das System konzentriert sich auf Queue-Based Load Leveling bzgl der Jobverarbeitung. Es werden zwar Clientseitige Retries durchgeführt, das ist aber keine Garantie, dass die Nachricht trotz hoher Last ankommt

bietet sich vor allem an, wenn man sowieso schon eine db nutzt, man braucht keine zusätzliche Infrastruktur (im Vergleich zum Message Broker, der als eigener Server läuft)

Bei sehr großen Systemen kann die Datenbank aufgrund einer hohen Anzahl Zugriffsversuche jedoch zum limitierenden Faktor werden, sodass dedizierte Messaging-Systeme hinsichtlich des erreichbaren Durchsatzes Vorteile bieten (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“).

vermeidung komplexität verteilter Transaktionen durch Nutzung lokaler ACID-Transaktionen da alles in derselben Datenbank liegt

Einschränkung Z-7 nur bis zu einem bestimmten Grad -> Kein Job kann verloren gehen, aber ein job wird nicht unendlich oft recovered da sonst Konflikt mit Z-9

8 Fazit und Ausblick

To be done

Literaturverzeichnis

- Apache Software Foundation (Hrsg.) (22. Mai 2026). *Apache Kafka Documentation 4.3.X*. AK 4.3.X. URL: <https://kafka.apache.org/43/> (besucht am 26.06.2026).
- Bass, Len (Aug. 2021). *Software architecture in practice*. Unter Mitarb. von Paul Clements und Rick Kazman. Fourth edition. SEI series in software engineering. Boston: Addison-Wesley.
- Broadcom Inc. (Hrsg.) (15. Juni 2026a). *RabbitMQ Documentation / RabbitMQ*. URL: <https://www.rabbitmq.com/docs> (besucht am 26.06.2026).
- (24. Mai 2026b). *Spring Boot*. Spring Boot. URL: <https://spring.io/projects/spring-boot> (besucht am 24.05.2026).
- Hangfire OÜ (Hrsg.) (26. Juni 2026). *Hangfire – Background jobs and workers for .NET and .NET Core*. URL: <https://www.hangfire.io> (besucht am 26.06.2026).
- Hohpe, Gregor und Bobby Woolf (2005). *Enterprise integration patterns designing, building, and deploying messaging solutions*. 5. print. Boston [u.a.: Addison-Wesley.
- Kleppmann, Martin (2017). *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. First edition. Beijing, [China: O'Reilly.
- Kudraß, Thomas (5. März 2015). *Taschenbuch Datenbanken*. Hrsg. von Kudraß Thomas. Carl Hanser Verlag GmbH & Co. KG. ISBN: 978-3-446-43508-7. DOI: 10.3139/9783446440265.fm. URL: <https://www.hanser-elibrary.com/doi/10.3139/9783446440265.fm> (besucht am 21.06.2026).
- Microsoft Corporation (Hrsg.) (31. März 2026a). *Best Practices for Background Jobs - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/background-jobs> (besucht am 17.06.2026).
- (12. Juni 2026b). *Queue-Based Load Leveling Pattern - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling> (besucht am 17.06.2026).
- Newman, Sam (Sep. 2026). *Building Resilient Distributed Systems*. O'Reilly. URL: <https://learning.oreilly.com/library/view/building-resilient-distributed/9781098163532/> (besucht am 24.05.2026).
- Richardson, Chris (2018). *Microservices patterns*. Software development. Shelter Island: Manning.
- Rosoco BV (Hrsg.) (24. Mai 2026). *JobRunr Documentation*. URL: <https://www.jobrunr.io/en/documentation/> (besucht am 24.05.2026).
- Saake, Gunter, Andreas Heuer und Kai-Uwe Sattler (Juni 2018). *Datenbanken – Konzepte und Sprachen*. mitp Verlags GmbH & Co KG. ISBN: 978-3-95845-778-2. URL: <https://learning.oreilly.com/library/view/datenbanken-konzepte/9783958457782/> (besucht am 19.06.2026).
- Temporal Technologies Inc. (24. Mai 2026). *Temporal Docs*. URL: <https://docs.temporal.io/> (besucht am 24.05.2026).
- Terracotta Inc. (Hrsg.) (24. Mai 2026). *Quartz Scheduler Documentation*. URL: <https://www.quartz-scheduler.org/documentation/quartz-2.5.x/> (besucht am 24.05.2026).

The PostgreSQL Global Development Group (Hrsg.) (14. Mai 2026). *PostgreSQL 18.4 Documentation*. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/18/index.html> (besucht am 21.06.2026).